

Hierarchical k -GNN Explanations via Generative Interpretation Networks

Comparing 1-GNN and 1-2-GNN for Model-Level
Graph Classification Interpretability

[Author Name]

March 2026

Abstract

Graph Neural Networks (GNNs) are powerful tools for graph classification, yet their decision processes remain opaque. Higher-order k -GNNs, which operate on k -element subsets of nodes, provably increase expressive power beyond the Weisfeiler–Leman hierarchy, but it is unclear whether this additional expressiveness leads the model to learn qualitatively different internal representations. We investigate this question by comparing a standard 1-GNN with a hierarchical 1-2-GNN using the GIN-Graph framework for model-level explanation generation. Our pipeline trains k -GNN classifiers on the MUTAG and PROTEINS benchmarks, then uses a Wasserstein GAN with gradient penalty to generate class-representative explanation graphs guided by the pretrained classifier. We introduce a fully differentiable dense wrapper that maintains gradient flow through both node features and adjacency matrices for hierarchical models. On MUTAG, both models achieve identical classification accuracy (89.5%), yet they generate different explanations: the 1-GNN associates mutagenicity with nitrogen-containing structures while the 1-2-GNN favours halogenated compounds, indicating that the two architectures encode different structural features of the same class. Cross-model classification of 500 generated graphs per class confirms this divergence quantitatively. On PROTEINS, 1-GNN-generated Non-Enzyme graphs are rejected by the 1-2-GNN (9.4% agreement), while 1-2-GNN-generated graphs are accepted by both models (98–100% agreement), indicating that the 1-2-GNN learns a more selective decision boundary in its pairwise feature space. These results demonstrate that different GNN architectures develop qualitatively different internal representations of graph classes, and that model-level explanations can reveal these differences even when classification accuracy is comparable.

Contents

1	Introduction	4
2	Background	5
2.1	Graph Neural Networks	5
2.2	The k -WL Hierarchy and k -GNNs	5
2.3	GIN-Graph	6
3	Method	6
3.1	1-GNN Architecture	6
3.2	2-GNN and the Hierarchical 1-2-GNN	7
3.3	GIN-Graph Generator	8
3.4	Training Objective	9
3.4.1	The Discriminator	9
3.4.2	The Two-Player Setup	9
3.4.3	Discriminator Loss	10
3.4.4	Generator Loss	10
3.4.5	Why WGAN-GP and Not a Vanilla GAN	11
3.4.6	GNN Guidance Loss	11
3.4.7	Dynamic Weighting	12
3.4.8	Structural Regularisation	12
3.5	Dense Wrapper for Differentiable k -GNN Inference	14
3.5.1	The Sparse Implementation	14
3.5.2	The Problem the Generator Creates	15
3.5.3	The Fix	15
3.5.4	Dense 1-GNN	16
3.5.5	Dense 2-GNN	16
3.5.6	Summary	17
3.6	Validation Score	17
3.6.1	Prediction Probability (p)	18
3.6.2	Embedding Similarity (s)	18
3.6.3	Degree Score (d)	19
3.6.4	Validity Threshold	19
3.6.5	Granularity (κ)	20
3.6.6	Evaluation Protocol	20
4	Experimental Setup	20

4.1	Datasets	20
4.2	Model Configuration	21
4.3	Evaluation Protocol	21
5	Results	22
5.1	k -GNN Classification Performance	22
5.2	GIN-Graph Explanation Quality: MUTAG	22
5.3	GIN-Graph Explanation Quality: PROTEINS	23
5.4	Metric Decomposition	25
5.5	Training Dynamics	28
5.6	Generated Dataset Comparison	28
5.6.1	Structural Fidelity	28
5.6.2	Cross-Model Classification	30
5.6.3	Embedding Space Structure	31
6	Discussion	33
6.1	Different Architectures, Different Representations	33
6.2	Qualitative Differences in Generated Explanations	33
6.3	Local Structure vs. Population Fidelity	34
6.4	The Granularity Problem	34
6.5	Limitations	35
6.6	Future Work	36
7	Conclusion	36

1 Introduction

Graph Neural Networks (GNNs) have become the standard approach for learning on graph-structured data, achieving strong results on tasks ranging from molecular property prediction to social network analysis. Despite their effectiveness, GNNs suffer from the same interpretability challenges as other deep learning models: their predictions are difficult to explain in human-understandable terms.

The expressiveness of standard message-passing GNNs is bounded by the 1-dimensional Weisfeiler–Leman (1-WL) graph isomorphism test [Morris et al.(2019)]. Morris et al. [Morris et al.(2019)] proposed k -GNNs that operate on k -element subsets of nodes, achieving expressiveness equivalent to the k -WL test. In theory, higher-order k -GNNs can distinguish graph structures that standard GNNs cannot.

A natural question arises: *does this additional structural expressiveness lead the model to learn different internal representations of graph classes, and can we observe these differences through generated explanations?* If a model captures richer structural patterns, the explanations it produces (representative graphs that characterise each class) may reveal what structural features each architecture has learned to associate with each class.

We investigate this question using the GIN-Graph framework [Sun et al.(2025)], which generates model-level explanation graphs through a GAN-based approach guided by a pretrained classifier. Model-level explanations aim to answer the question “what does a typical graph of class c look like according to the model?” by generating new graphs that maximise the model’s confidence for that class while remaining structurally realistic. This contrasts with instance-level methods that explain individual predictions by identifying important subgraphs.

Specifically, we compare:

- A **1-GNN**: standard node-level message passing, equivalent to 1-WL;
- A **1-2-GNN**: hierarchical architecture combining node-level (1-GNN) and pairwise (2-GNN) message passing, achieving expressiveness beyond 1-WL.

This work:

1. Reuses the dense 1-GNN wrapper from GIN-Graph [Sun et al.(2025)] and applies the same logic to the 2-GNN branch of a 1-2-GNN classifier, so that GIN-Graph training can be run against a hierarchical k -GNN.
2. Computes the embedding similarity component s of the validation score via cosine similarity between generated and real graph embeddings. The GIN-Graph reference implementation uses the prediction probability p as a proxy for s ; we use the actual embedding.
3. Compares the explanations produced by the two architectures on MUTAG and PROTEINS. The two models produce different explanation structures and show asymmetric cross-model classification agreement, with some configurations showing near-zero agreement.

2 Background

2.1 Graph Neural Networks

A graph $G = (V, E)$ consists of a node set V with $|V| = n$ and an edge set $E \subseteq V \times V$. Each node $v \in V$ carries a feature vector $\mathbf{x}_v \in \mathbb{R}^d$. We denote the adjacency matrix as $\mathbf{A} \in \{0, 1\}^{n \times n}$ where $A_{uv} = 1$ if $(u, v) \in E$, and the feature matrix as $\mathbf{X} \in \mathbb{R}^{n \times d}$ where row v is $\mathbf{x}_v$.

GNNs learn node representations through iterative *message passing*. At each layer t , a node updates its representation by aggregating information from its neighbours:

$$\mathbf{h}_v^{(t)} = \text{UPDATE}(\mathbf{h}_v^{(t-1)}, \text{AGGREGATE}(\{\{\mathbf{h}_u^{(t-1)} : u \in \mathcal{N}(v)\}\})), \quad (1)$$

where $\mathcal{N}(v) = \{u \in V : (u, v) \in E\}$ denotes the neighbourhood of v , $\mathbf{h}_v^{(0)} = \mathbf{x}_v$ is the initial node feature, and $\{\{\cdot\}\}$ denotes a multiset.

The choice of UPDATE and AGGREGATE functions determines the specific GNN variant. In our implementation, these are parameterised by separate linear transformations for the self-feature and the aggregated neighbour features. After T layers of message passing, a *readout* function pools all node embeddings into a single graph-level representation:

$$\mathbf{h}_G = \text{READOUT}(\{\mathbf{h}_v^{(T)} : v \in V\}). \quad (2)$$

Common choices for READOUT include sum, mean, and max pooling over the node set. We use sum pooling for the classifier readouts, as it preserves information about both the features and the number of nodes.

2.2 The k -WL Hierarchy and k -GNNs

The *Weisfeiler–Leman* (WL) graph isomorphism test is a classical algorithm that iteratively refines node colour labels based on the multiset of neighbour colours. Two graphs are distinguished if, after some number of iterations, they produce different multisets of colours. The 1-WL test operates on individual nodes.

A fundamental result in GNN theory is that standard message-passing GNNs are *at most* as powerful as the 1-WL test [Morris et al.(2019)]: if 1-WL cannot distinguish two graphs, no message-passing GNN can either. This means there exist structurally different graphs (e.g., certain regular graphs) that no standard GNN can tell apart.

The k -WL test generalises this by operating on k -tuples of nodes, achieving strictly increasing discriminative power: for all $k \geq 2$, $(k+1)$ -WL is strictly more powerful than k -WL [Cai et al.(1992)]. Morris et al. [Morris et al.(2019)] proposed k -GNNs that achieve this higher expressiveness by performing message passing on k -element subsets of nodes (“ k -sets”) rather than individual nodes.

For a k -set $s = \{v_1, \dots, v_k\} \subseteq V$, the *local neighbourhood* is:

$$\mathcal{N}_L(s) = \{s' \subseteq V : |s'| = k, |s \Delta s'| = 2, \text{ the differing elements are adjacent in } G\}, \quad (3)$$

where $s \Delta s'$ denotes the symmetric difference. In words: a neighbour of s is obtained by replacing exactly one element of s with a new node that is adjacent (in the original graph G) to the removed node.

Example. Consider a 2-set $s = \{u, v\}$. Its neighbours are all pairs $\{v, w\}$ where $(u, w) \in E$ (replacing u with adjacent w) and all pairs $\{u, w\}$ where $(v, w) \in E$ (replacing v with adjacent w). This means the 2-GNN can capture pairwise relationships and bond-level patterns that are invisible to a 1-GNN operating on individual nodes.

2.3 GIN-Graph

GIN-Graph [Sun et al.(2025)] is a model-level explanation method that generates class-representative graphs using a generative adversarial approach. The key idea is to train a generator $\mathcal{G}$ to produce graphs that satisfy two objectives simultaneously:

1. **Realism:** generated graphs should be structurally similar to real graphs from the dataset, enforced by a WGAN-GP discriminator.
2. **Class specificity:** generated graphs should be confidently classified as the target class by the pretrained GNN, enforced by a classification guidance loss.

The generator maps noise $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ to a graph $(\tilde{A}, \tilde{X})$ using Gumbel-Softmax for differentiable discrete sampling. The training objective balances the two losses through a dynamic weighting schedule that shifts focus from realism (early training) to class specificity (late training).

3 Method

This section details the mathematical formulations of our two classifier architectures (1-GNN and 1-2-GNN), the GIN-Graph generator, the training procedure, and the differentiable dense wrapper that bridges them.

3.1 1-GNN Architecture

Our 1-GNN implements message passing with separate transformations for self-features and neighbour-aggregated features. At layer t , the update rule for node v is:

$$\mathbf{h}_v^{(t)} = \sigma \left(\mathbf{h}_v^{(t-1)} \mathbf{W}_1^{(t)} + \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(t-1)} \mathbf{W}_2^{(t)} \right), \quad (4)$$

where $\mathbf{W}_1^{(t)}, \mathbf{W}_2^{(t)} \in \mathbb{R}^{d_{t-1} \times d_t}$ are learnable weight matrices, σ is the ReLU activation function $\sigma(x) = \max(0, x)$, and $d_0 = d$ (the input feature dimension). The self-transformation $\mathbf{h}_v^{(t-1)} \mathbf{W}_1^{(t)}$ allows the model to retain and transform the node’s own representation, while the neighbour term $\sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(t-1)} \mathbf{W}_2^{(t)}$ aggregates structural context.

This can equivalently be written in matrix form for the entire graph:

$$\mathbf{H}^{(t)} = \sigma \left(\mathbf{H}^{(t-1)} \mathbf{W}_1^{(t)} + \mathbf{A} \mathbf{H}^{(t-1)} \mathbf{W}_2^{(t)} \right), \quad (5)$$

where $\mathbf{H}^{(t)} \in \mathbb{R}^{n \times d_t}$ is the matrix of all node representations at layer t , and $\mathbf{A} \in \{0, 1\}^{n \times n}$ is the adjacency matrix. The matrix product $\mathbf{A} \mathbf{H}^{(t-1)}$ computes the sum of neighbour features for every node simultaneously.

After $T = 3$ layers with hidden dimension $d_1 = d_2 = d_3 = 64$, the graph embedding is obtained via sum pooling:

$$\mathbf{h}_G^{(1)} = \sum_{v \in V} \mathbf{h}_v^{(T)} \in \mathbb{R}^{d_T}. \quad (6)$$

This embedding is fed through a two-layer classifier MLP with ReLU activation and dropout:

$$\hat{y} = \text{softmax}\left(\mathbf{W}_o \sigma(\mathbf{W}_c \mathbf{h}_G^{(1)})\right), \quad (7)$$

where $\mathbf{W}_c \in \mathbb{R}^{d_T \times d_T}$ and $\mathbf{W}_o \in \mathbb{R}^{d_T \times C}$ with C being the number of classes.

3.2 2-GNN and the Hierarchical 1-2-GNN

The 2-GNN operates on 2-sets (unordered node pairs) rather than individual nodes. For each pair $s = \{u, v\}$ with $u < v$ (canonical ordering), the initial feature vector incorporates the 1-GNN embeddings of both nodes and an *isomorphism type* indicator:

$$\mathbf{f}_s^{(0)} = [\mathbf{h}_u^{(T)} \parallel \mathbf{h}_v^{(T)} \parallel \mathbb{I}[(u, v) \in E]] \in \mathbb{R}^{2d_T+1}, \quad (8)$$

where $\parallel$ denotes vector concatenation. The isomorphism type $\mathbb{I}[(u, v) \in E] \in \{0, 1\}$ indicates whether u and v are connected by an edge. This allows the 2-GNN to distinguish between pairs of connected nodes and pairs of unconnected nodes, a distinction that carries structural meaning (e.g., in a molecule, bonded vs. non-bonded atom pairs).

Message passing on 2-sets follows the neighbourhood definition from Equation (3). For a 2-set $s = \{u, v\}$, the local neighbourhood decomposes into two cases:

$$\mathcal{N}_L(\{u, v\}) = \underbrace{\{\{v, w\} : w \neq u, (u, w) \in E\}}_{\text{Case 1: replace } u \text{ with } w} \cup \underbrace{\{\{u, w\} : w \neq v, (v, w) \in E\}}_{\text{Case 2: replace } v \text{ with } w}. \quad (9)$$

In Case 1, we remove u from the pair and add a new node w that must be adjacent to u in the original graph. In Case 2, we remove v and add w adjacent to v . This ensures that neighbourhood relationships in the 2-GNN reflect the edge structure of the underlying graph.

The 2-GNN layer update rule mirrors the 1-GNN structure:

$$\mathbf{f}_s^{(\ell+1)} = \sigma \left(\mathbf{f}_s^{(\ell)} \mathbf{W}_3^{(\ell)} + \sum_{s' \in \mathcal{N}_L(s)} \mathbf{f}_{s'}^{(\ell)} \mathbf{W}_4^{(\ell)} \right), \quad (10)$$

where $\mathbf{W}_3^{(\ell)}, \mathbf{W}_4^{(\ell)}$ are the 2-GNN weight matrices at layer ℓ . We apply $T_2 = 2$ layers.

The **hierarchical 1-2-GNN** combines both levels. It first runs the 1-GNN (Equation 4) for $T_1 = 3$ layers to obtain node embeddings, then uses these embeddings to initialise 2-GNN features (Equation 8), and runs $T_2 = 2$ layers of 2-GNN message passing. The final graph representation concatenates both readouts:

$$\mathbf{h}_G = [\mathbf{h}_G^{(1)} \parallel \mathbf{h}_G^{(2)}] \in \mathbb{R}^{2d_T}, \quad (11)$$

where the 1-GNN readout is $\mathbf{h}_G^{(1)} = \sum_{v \in V} \mathbf{h}_v^{(T_1)}$ and the 2-GNN readout pools over all pairs:

$$\mathbf{h}_G^{(2)} = \sum_{\{u, v\} \in \binom{V}{2}} \mathbf{f}_{\{u, v\}}^{(T_2)} \in \mathbb{R}^{d_T}. \quad (12)$$

The concatenated embedding $\mathbf{h}_G$ is then fed through the same classifier structure as Equation (7), but with input dimension $2d_T$ instead of d_T .

Scalability. The number of 2-sets grows as $\binom{n}{2} = \frac{n(n-1)}{2}$, which becomes prohibitive for large graphs. For PROTEINS (up to 620 nodes, yielding up to 191,890 pairs), we process all pairs using a numba-accelerated sparse kernel with CSR adjacency representation, achieving efficient $O(\text{degree})$ neighbour lookup instead of $O(N)$ dense masks. This enables exact computation over the full pairwise structure without sampling.

3.3 GIN-Graph Generator

The generator $\mathcal{G}$ maps a latent noise vector $\mathbf{z} \in \mathbb{R}^{d_z}$ to a graph $(\tilde{A}, \tilde{X})$ through a backbone MLP followed by separate output heads:

$$\mathbf{h} = \text{LeakyReLU}(\mathbf{W}_2 \text{LeakyReLU}(\mathbf{W}_1 \mathbf{z})) \in \mathbb{R}^{2d_h}, \quad (13)$$

$$\tilde{A}_{\text{raw}} = \text{sym}(\text{reshape}(\mathbf{h} \mathbf{W}_A, N \times N)), \quad (14)$$

$$\tilde{X}_{\text{raw}} = \text{reshape}(\mathbf{h} \mathbf{W}_X, N \times D), \quad (15)$$

where $d_z = 32$ is the latent dimension, $d_h = 128$ is the hidden dimension, N is the maximum number of nodes, D is the number of node feature types, and $\text{sym}(\mathbf{M}) = \frac{1}{2}(\mathbf{M} + \mathbf{M}^\top)$ ensures the adjacency logits are symmetric (undirected graphs).

Gumbel-Softmax. Graphs are inherently discrete objects: edges are either present or absent, and node types are categorical. Standard gradient-based optimisation cannot backpropagate through discrete sampling. The Gumbel-Softmax trick [Jang et al.(2017)] provides a differentiable relaxation.

For a categorical distribution with unnormalised log-probabilities (logits) $\pi_1, \dots, \pi_K$, the Gumbel-Softmax sample is:

$$y_i = \frac{\exp((\pi_i + g_i)/\tau)}{\sum_{j=1}^K \exp((\pi_j + g_j)/\tau)}, \quad g_i = -\log(-\log(u_i)), \quad u_i \sim \text{Uniform}(0, 1), \quad (16)$$

where $\tau > 0$ is a temperature parameter. As $\tau \rightarrow 0$, the output approaches a one-hot vector (hard categorical sample); as $\tau \rightarrow \infty$, it approaches a uniform distribution. During training, we use the *straight-through estimator*: the forward pass produces hard (discrete) samples via $\arg \max$, but the backward pass uses the continuous Gumbel-Softmax gradients. This gives the generator discrete outputs while maintaining gradient flow.

For the **adjacency matrix**, each potential edge (i, j) is a binary choice (edge/no-edge). We construct 2-class logits $[\tilde{A}_{\text{raw}}[i, j], -\tilde{A}_{\text{raw}}[i, j]]$ and apply Gumbel-Softmax with $K = 2$. To guarantee symmetry ($\tilde{A}_{ij} = \tilde{A}_{ji}$), we sample only the upper triangle and mirror it:

$$\tilde{A}_{\text{upper}} = \text{triu}(\text{GumbelSoftmax}(\tilde{A}_{\text{raw}}, \tau)), \quad \tilde{A} = \tilde{A}_{\text{upper}} + \tilde{A}_{\text{upper}}^\top. \quad (17)$$

For **node features**, each node has D possible types (e.g., 7 atom types for MUTAG). We apply Gumbel-Softmax with $K = D$ to produce one-hot feature vectors:

$$\tilde{X}[v, :] = \text{GumbelSoftmax}(\tilde{X}_{\text{raw}}[v, :], \tau) \in \{0, 1\}^D. \quad (18)$$

The temperature $\tau = 1.0$ during training (allowing exploration) and $\tau = 0.1$ during evaluation (producing sharper, more discrete outputs).

3.4 Training Objective

The total generator loss combines adversarial, GNN-guidance, and structural regularisation terms:

$$\mathcal{L}_G = (1 - \lambda(t)) \mathcal{L}_{\text{GAN}} + \lambda(t) \mathcal{L}_{\text{GNN}} + \mathcal{L}_{\text{reg}}, \quad (19)$$

where $\lambda(t) \in [0, 1]$ is a dynamic weight that increases over training, and $\mathcal{L}_{\text{reg}}$ comprises structural regularisation losses described below. The three terms correspond to three goals: making the generated graphs look realistic ($\mathcal{L}_{\text{GAN}}$), making them belong to the target class ($\mathcal{L}_{\text{GNN}}$), and making them the right size and composition ($\mathcal{L}_{\text{reg}}$). We now explain each in turn, beginning with the adversarial component and the discriminator that drives it.

3.4.1 The Discriminator

The discriminator $\mathcal{D}$ is a graph neural network that maps a graph to a single scalar. It operates on dense batched inputs (node features $\mathbf{X} \in \mathbb{R}^{B \times N \times D}$ and adjacency $\mathbf{A} \in [0, 1]^{B \times N \times N}$) so that the same forward pass handles both real and generated graphs. The architecture has three stages:

1. **Two dense GCN layers** with hidden dimension 128 and ReLU activations. Each layer computes $\mathbf{H}^{(t)} = \sigma(\hat{\mathbf{A}} \mathbf{H}^{(t-1)} \mathbf{W}^{(t)})$, where $\hat{\mathbf{A}}$ is the adjacency with self-loops added and symmetric degree normalisation applied.
2. **Mean pooling** across the node dimension: $\mathbf{h}_G = \frac{1}{N} \sum_{v=1}^N \mathbf{h}_v^{(2)} \in \mathbb{R}^{128}$. This produces a single graph-level vector per graph.
3. **Two-layer MLP** with LeakyReLU activation: $\mathcal{D}(G) = \mathbf{w}^\top \text{LeakyReLU}_{0.2}(\mathbf{W} \mathbf{h}_G) \in \mathbb{R}$.

Higher values of $\mathcal{D}(G)$ indicate that the graph appears real; lower values indicate that it appears generated. There is no sigmoid or softmax at the output, so $\mathcal{D}(G)$ is not a probability. In a standard binary classifier, the scalar would pass through a sigmoid $\sigma(x) = 1/(1 + e^{-x})$ to produce a value in $[0, 1]$. Here the output is left unbounded. The reason is explained below in the comparison with vanilla GANs.

The discriminator is distinct from the pretrained k -GNN classifier Φ used in $\mathcal{L}_{\text{GNN}}$: $\mathcal{D}$ is trained from scratch during GIN-Graph training and scores graphs on realism, while Φ is frozen and scores graphs on class membership.

3.4.2 The Two-Player Setup

The generator $\mathcal{G}$ and the discriminator $\mathcal{D}$ are trained in alternating steps:

- The **discriminator** receives a batch of real graphs from the training set and a batch of generated graphs from the current generator, and updates its weights to increase scores on real graphs and decrease scores on generated graphs.
- The **generator** produces a batch of graphs, passes them through the discriminator, and updates its weights to increase the scores the discriminator assigns.

Neither network is trained to convergence; they co-evolve. In each training step, the discriminator updates once, then the generator updates once ($n_{\text{critic}} = 1$). This alternation is necessary. If $\mathcal{D}$ were trained to convergence against a fixed generator, its output would be correct (low on all generated graphs, high on all real graphs) but uninformative as a

gradient signal: the generated region of input space would become flat, so small changes to a generated graph would not change $\mathcal{D}$'s score, and the generator would receive no direction for improvement. Keeping $\mathcal{D}$ partially trained ensures that its scores still vary across the region where the generator's current outputs lie.

3.4.3 Discriminator Loss

With the setup in place, we can state the discriminator loss. In each training step, we draw a batch of $B = 64$ real graphs G from the training set and a batch of B fake graphs $\tilde{G}$ from the generator. The discriminator minimises:

$$\mathcal{L}_D = \underbrace{\mathbb{E}_{\tilde{G} \sim \mathcal{G}}[\mathcal{D}(\tilde{G})]}_{\text{score on fake}} - \underbrace{\mathbb{E}_{G \sim p_{\text{data}}}[\mathcal{D}(G)]}_{\text{score on real}} + \underbrace{\lambda_{\text{GP}} \mathbb{E}_{\tilde{G}}[(\|\nabla_{\tilde{G}} \mathcal{D}(\tilde{G})\|_2 - 1)^2]}_{\text{gradient penalty}}, \quad (20)$$

where each expectation $\mathbb{E}[\cdot]$ is in practice the average over one batch. Consider the first two terms:

- $\mathbb{E}_{\tilde{G} \sim \mathcal{G}}[\mathcal{D}(\tilde{G})]$ is the average score the discriminator assigns to generated graphs. The discriminator minimises $\mathcal{L}_D$, so it pushes this term down, assigning *low* scores to generated graphs.
- $-\mathbb{E}_{G \sim p_{\text{data}}}[\mathcal{D}(G)]$ is the negative of the average score on real graphs. Minimising this term pushes the average score on real graphs *up*.

Together, these two terms encode one instruction: push real scores up, push generated scores down. The third term, the gradient penalty, is a stability device explained below.

3.4.4 Generator Loss

The generator's adversarial loss is the negation of the discriminator's "score on fake" term:

$$\mathcal{L}_{\text{GAN}} = -\mathbb{E}_{\tilde{G} \sim \mathcal{G}}[\mathcal{D}(\tilde{G})]. \quad (21)$$

The generator wants to maximise the score the discriminator assigns to its outputs. Since optimisers minimise by convention, we negate: minimising $-\mathbb{E}[\mathcal{D}(\tilde{G})]$ is equivalent to maximising $\mathbb{E}[\mathcal{D}(\tilde{G})]$.

Real graphs do not appear in $\mathcal{L}_{\text{GAN}}$. The generator never sees real data directly. It learns about the real distribution only through the gradient signal from $\mathcal{D}$: real graphs train $\mathcal{D}$, and $\mathcal{D}$'s gradient informs the generator how to adjust its outputs. This indirect chain, from real data to $\mathcal{D}$'s parameters to $\mathcal{D}$'s gradient to the generator's parameters, is the core mechanism of GANs.

3.4.5 Why WGAN-GP and Not a Vanilla GAN

In a vanilla GAN, the discriminator ends in a sigmoid, so its output is a value in $[0, 1]$ interpreted as the probability of the input being real. The problem is gradient saturation. Once $\mathcal{D}$ can reliably separate real from generated graphs, the sigmoid output for generated graphs is pushed toward 0. The derivative of the sigmoid $\sigma(x) = 1/(1 + e^{-x})$ is $\sigma(x)(1 - \sigma(x))$, which approaches zero when $\sigma(x) \approx 0$ or $\sigma(x) \approx 1$. The generator's weight updates

are computed via the chain rule as a product of derivatives, one of which is this sigmoid derivative. A near-zero factor makes the entire product near-zero, and the generator stops learning. This is the core difficulty of vanilla GAN training.

WGAN removes the sigmoid. The discriminator’s output is an unbounded real number with no flat region. When $\mathcal{D}$ becomes strong, its output for generated graphs continues decreasing, and the derivative remains non-degenerate, so the generator continues to receive a gradient signal.

Removing the sigmoid introduces a new problem: without any bound on $\mathcal{D}$ ’s output, it can reduce its loss without limit by scaling its weights up. Doubling all weights roughly doubles the output gap between real and generated scores. The gradient penalty in Equation (20) prevents this. The term

$$\lambda_{\text{GP}} \mathbb{E}_{\hat{G}} \left[(\|\nabla_{\hat{G}} \mathcal{D}(\hat{G})\|_2 - 1)^2 \right]$$

penalises deviation of the gradient magnitude of $\mathcal{D}$ ’s output (with respect to its input $\hat{G}$) from 1. Here $\hat{G} = \epsilon G + (1 - \epsilon)\tilde{G}$ with $\epsilon \sim \text{Uniform}(0, 1)$ is a random interpolation between a real and a generated graph, so the penalty is evaluated at points between the two distributions. The coefficient $\lambda_{\text{GP}} = 10$.

This constraint has two effects. First, $\mathcal{D}$ cannot reduce its loss by weight rescaling, because rescaling increases the gradient magnitude and the penalty dominates. Second, the gradient flowing from $\mathcal{D}$ into the generator remains at a bounded magnitude, neither vanishing (the vanilla GAN failure) nor exploding (the unconstrained WGAN failure).

In summary: a vanilla GAN’s sigmoid saturates when $\mathcal{D}$ is confident, killing the generator’s gradient. WGAN-GP removes the sigmoid and adds a gradient penalty that keeps the gradient magnitude near 1. The generator therefore receives a non-degenerate learning signal regardless of $\mathcal{D}$ ’s strength.

3.4.6 GNN Guidance Loss

The adversarial loss enforces structural realism but does not steer generation toward any particular class. The GNN guidance loss maximises the pretrained classifier’s confidence on the target class c :

$$\mathcal{L}_{\text{GNN}} = -\log p_{\Phi}(y = c \mid \tilde{G}), \quad (22)$$

where $p_{\Phi}(y = c \mid \tilde{G})$ is the softmax probability that the pretrained k -GNN Φ assigns to class c for the generated graph $\tilde{G}$. This is the negative log-likelihood of the target class. The classifier’s weights are frozen; only the generator’s weights are updated. Minimising $\mathcal{L}_{\text{GNN}}$ pushes the generator to produce graphs that the classifier assigns to class c .

3.4.7 Dynamic Weighting

The balance parameter $\lambda(t)$ in Equation (19) follows a sigmoid schedule [Sun et al.(2025)]:

$$\lambda(t) = \lambda_{\min} + (\lambda_{\max} - \lambda_{\min}) \cdot \sigma \left(k \cdot \left(\frac{2(t/T - p)}{1 - p} - 1 \right) \right), \quad (23)$$

where T is the total number of training iterations ($T = 300$ epochs in our experiments), $p = 0.4$ is the fraction of training before λ begins increasing, $k = 10$ controls the steepness of the transition, and $\sigma(x) = 1/(1 + e^{-x})$ is the logistic sigmoid.

When $t \ll pT$ (the first ~ 120 epochs), the sigmoid argument is strongly negative, giving $\lambda(t) \approx 0$: the total loss is $\mathcal{L}_G \approx \mathcal{L}_{\text{GAN}}$, and the generator focuses on producing structurally realistic graphs without class-specific pressure. As t passes pT , $\lambda(t)$ increases toward 1: the total loss shifts to $\mathcal{L}_G \approx \mathcal{L}_{\text{GNN}}$, directing the generator toward class-specific patterns. Without this schedule, the generator collapses to structurally degenerate graphs that satisfy the classifier but bear no resemblance to real data, since $\mathcal{L}_{\text{GNN}}$ can be minimised by any graph that triggers the correct class output, regardless of structural plausibility.

3.4.8 Structural Regularisation

The generator outputs a fixed-size $N \times N$ adjacency matrix and $N \times D$ feature matrix ($N = 28$ for MUTAG, $N = 50$ for PROTEINS). Without explicit constraints, the Gumbel-Softmax sampling activates most slots and settles on atom-type distributions that satisfy the classifier but do not resemble real class members. $\mathcal{L}_{\text{GAN}}$ constrains these statistics only indirectly through the discriminator, and the signal is too weak to prevent these failures. The regularisation term $\mathcal{L}_{\text{reg}}$ supplies direct supervision on two graph-level statistics: graph size and atom-type frequencies. Both targets are precomputed once from the real training set and remain fixed during training.

The regulariser decomposes as $\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{size}} + \mathcal{L}_{\text{type}}$, combined into the full expression:

$$\mathcal{L}_{\text{reg}} = \lambda_{\text{size}} \cdot \left(\frac{\hat{n} - \bar{n}_c}{\bar{n}_c} \right)^2 + \lambda_{\text{type}} \cdot \sum_{i=1}^D (\hat{p}_i - p_i^c)^2, \quad (24)$$

where $\mathcal{L}_{\text{size}}$ penalises deviation from the class mean graph size and $\mathcal{L}_{\text{type}}$ penalises deviation from the class atom-type distribution. We now derive each term.

Size penalty. The generator outputs a fixed $N \times N$ matrix with no flag marking which slots are active. A slot is effectively active if it has at least one edge to another slot, so the natural notion of graph size is the number of slots with at least one edge.

Define the soft degree of slot v as the row-sum of the soft adjacency matrix:

$$d_v = \sum_{u=1}^N \tilde{A}_{vu}.$$

A slot with no edges has $d_v \approx 0$; a slot with one edge has $d_v \approx 1$; a slot with three edges has $d_v \approx 3$. The naive active-slot count would use an indicator:

$$\hat{n}_{\text{naive}} = \sum_{v=1}^N \mathbb{I}[d_v > 0.5],$$

treating a slot as active if it has at least half an edge of connection. This does not work for training: the indicator is a step function with derivative zero almost everywhere and undefined at the step. Any gradient flowing from $\mathcal{L}_{\text{size}}$ through this path collapses to zero.

We therefore replace the step with a smooth approximation. The logistic sigmoid $\sigma(x) = 1/(1 + e^{-x})$ approximates a step at $x = 0$. To place the transition at $d_v = 0.5$ and sharpen it, we shift the argument by 0.5 and scale by 10:

$$\hat{n} = \sum_{v=1}^N \sigma(10 \cdot (d_v - 0.5)).$$

The multiplier 10 is steep enough that slots with $d_v < 0.3$ contribute near zero and slots with $d_v > 0.7$ contribute near one:

d_v	0	0.3	0.5	0.7	1.0
$\sigma(10(d_v - 0.5))$	0.007	0.12	0.50	0.88	0.99

The expression approximates the active-slot count on the forward pass while providing non-zero gradients everywhere on the backward pass.

With $\hat{n}$ (soft count of active slots) and $\bar{n}_c$ (mean node count of real class- c graphs, pre-computed once), the size penalty is

$$\mathcal{L}_{\text{size}} = \lambda_{\text{size}} \left(\frac{\hat{n} - \bar{n}_c}{\bar{n}_c} \right)^2.$$

Two design choices matter. The squared error is smooth at zero with derivative vanishing as the error approaches zero, so the generator settles into the target without oscillation; the absolute value has a non-differentiable corner at zero and a constant-magnitude gradient. The division by $\bar{n}_c$ converts the error into a relative error, making the penalty scale-invariant across datasets: a 5-node error on MUTAG class 0 ($\bar{n}_c \approx 13.9$) is a $\sim 36\%$ failure, while the same error on a dataset with $\bar{n}_c \approx 40$ is only $\sim 12.5\%$. Without this normalisation, λ_{size} would need to be retuned for every new dataset.

The coefficient $\lambda_{\text{size}} = 10$ is ten times larger than λ_{type} because a size violation breaks the graph structurally (a 50-node molecule where the class mean is 14 is unrecognisable), while atom-type violations are a softer failure.

Atom-type penalty. For each class c , the target distribution $p^c = (p_1^c, \dots, p_D^c)$ is the frequency of each atom type in real class- c graphs, where p_i^c is the fraction of atoms of type i . This is a probability vector: entries in $[0, 1]$ summing to 1. For MUTAG class 0, $p^c \approx (0.85, 0.08, 0.04, \dots)$ across the $D = 7$ atom types, with carbon dominating. Computed once, stored, reused.

The generated frequencies are computed from $\tilde{X}$. Each row of $\tilde{X}$ is nearly one-hot after Gumbel-Softmax, so summing column i counts how many slots selected atom type i :

$$\hat{p}_i = \frac{1}{N} \sum_{v=1}^N \tilde{X}_{v,i}.$$

Both $\hat{p}$ and p^c are probability distributions over the same D atom types. The penalty is the squared L2 distance between them, summed across types:

$$\mathcal{L}_{\text{type}} = \lambda_{\text{type}} \sum_{i=1}^D (\hat{p}_i - p_i^c)^2,$$

with $\lambda_{\text{type}} = 1$. The coefficient is smaller than λ_{size} because the sum already accumulates D squared errors and reaches meaningful magnitudes with a unit weight.

The natural distance between probability distributions is KL divergence, not L2. We use L2 for three reasons:

1. KL is undefined when $p_i^c = 0$, because the formula contains $\log(\hat{p}_i/p_i^c)$. On MUTAG, some classes contain zero instances of some atom types, so this is a concrete problem.
2. KL is asymmetric: $D_{\text{KL}}(\hat{p} \| p^c) \neq D_{\text{KL}}(p^c \| \hat{p})$. The two directions favour different failure modes and selecting one requires arbitrary justification. L2 is symmetric by construction.
3. KL has a gradient with a $1/p_i^c$ factor that explodes for rare atom types. The L2 gradient is $2(\hat{p}_i - p_i^c)$, bounded and stable.

Gradient path. $\mathcal{L}_{\text{reg}}$ has the shortest gradient path of the three loss components. Its gradients flow from scalar arithmetic on $(\tilde{A}, \tilde{X})$ through the Gumbel-Softmax backward pass into the generator MLP, without invoking the discriminator or the classifier. It is therefore the cheapest term per training step, since every other term requires a full network forward and backward pass while $\mathcal{L}_{\text{reg}}$ is closed-form. The target class enters only through the precomputed statistics $\bar{n}_c$ and p^c , so changing the target class requires no change to the computation graph.

3.5 Dense Wrapper for Differentiable k -GNN Inference

This section explains why the pretrained classifier cannot be used directly inside the generator’s training loop, and how we solve this. The 1-GNN dense wrapper is taken from GIN-Graph [Sun et al.(2025)]; the same logic is applied to the 2-GNN branch of the 1-2-GNN classifier.

3.5.1 The Sparse Implementation

The classifier is trained on discrete, sparse graphs. A MUTAG molecule has around 20 edges out of $28 \times 28 = 784$ possible adjacency entries. Representing the graph as a full $N \times N$ matrix and computing $\mathbf{A} \mathbf{H}^{(t-1)}$ as a dense matrix product would waste computation on the zero entries.

The classifier instead uses PyTorch Geometric’s sparse representation. A graph is stored as an `edge_index` tensor of shape $[2, |E|]$: a list of edges where the first row holds source indices and the second row holds destination indices. For a MUTAG molecule with 20 edges (40 directed entries for an undirected graph), this tensor has 40 columns instead of 784 matrix entries. Message passing is done via `scatter_add`: for each edge (i, j) , the feature vector of node j is transformed by $\mathbf{W}_2^{(t)}$ and added to an accumulator for node i . After processing all edges, the accumulator for each node contains the sum of its neighbours’ transformed features, which is the same result as $\mathbf{A} \mathbf{H}^{(t-1)} \mathbf{W}_2^{(t)}$, without materialising $\mathbf{A}$.

The classifier’s forward function has the signature `forward(x, edge_index)`. It does not accept an adjacency matrix. For classifier training this is appropriate: the adjacency is a fixed discrete input, no gradients flow through it, and the sparse form is faster.

Two points matter for what follows: (i) the sparse form computes exactly the same function as the dense matrix form on discrete graphs, so it is an implementation choice, not a modelling choice; and (ii) the sparse form was chosen for speed and memory reasons that no longer apply inside the generator’s training loop.

3.5.2 The Problem the Generator Creates

The generator outputs a continuous adjacency matrix $\tilde{A} \in [0, 1]^{N \times N}$, where each entry is a soft edge weight produced by the Gumbel-Softmax straight-through estimator (Section 3.3). Training requires backpropagating from $\mathcal{L}_{\text{GNN}} = -\log p_{\Phi}(y = c \mid \tilde{A}, \tilde{X})$ (Equation 22) to the generator’s weights. This requires the partial derivative $\partial f_{\Phi} / \partial \tilde{A}$, the gradient of the classifier’s output with respect to its adjacency input.

The classifier’s forward function does not take an adjacency matrix. It takes an `edge_index`. Converting $\tilde{A}$ to an edge list requires thresholding: keep entries where $\tilde{A}_{ij} > 0.5$, discard the rest. Thresholding is a step function. Its derivative is zero everywhere (and undefined at the step). After thresholding, the `edge_index` consists of integer indices with no autograd connection to the continuous values they came from. The partial derivative $\partial f_{\Phi} / \partial \tilde{A}$ becomes zero. The generator receives no learning signal from $\mathcal{L}_{\text{GNN}}$.

The failure is mechanical. The chain rule computes a product of derivatives along the path from $\tilde{A}$ to $\mathcal{L}_{\text{GNN}}$. One factor in that product is the derivative of the thresholding step, which is zero. A zero factor makes the entire product zero, and the generator’s weights do not update.

3.5.3 The Fix

The fix is to rewrite the classifier’s forward pass to consume $\tilde{A}$ directly as a continuous matrix. The mathematical function is unchanged: on a discrete $\{0, 1\}$ adjacency matrix, the dense rewrite returns the same output as the sparse version. The implementation differs: where the sparse version uses `scatter_add` on an edge list, the dense version uses matrix multiplication $\tilde{A} \mathbf{H}^{(t-1)}$ via `torch.matmul`. Autograd can differentiate matrix multiplication: if $\mathbf{Y} = \tilde{A} \mathbf{H}$, then $\partial \mathbf{Y} / \partial \tilde{A}_{ij} = \mathbf{H}[j, :]$, and every entry of $\tilde{A}$ has a well-defined derivative. The chain rule no longer contains a zero factor.

The pretrained weights $\mathbf{W}_1^{(t)}, \mathbf{W}_2^{(t)}, \mathbf{W}_3^{(\ell)}, \mathbf{W}_4^{(\ell)}$ are copied from the sparse model unchanged. These weights parameterise the update rule (how features are transformed and combined), not the implementation (how edges are iterated). They are valid in both code paths and remain frozen throughout GIN-Graph training; only the generator’s weights are updated.

The dense wrapper is not a different model. It is the same model with a different forward function that accepts continuous adjacency matrices and maintains a gradient path from $\tilde{A}$ to the classifier’s output.

3.5.4 Dense 1-GNN

For the 1-GNN branch, the rewrite is direct. The node-level update (Equation 5) translates to dense batched computation. For a batch of $B = 64$ graphs with node features $\mathbf{X} \in \mathbb{R}^{B \times N \times d}$ and adjacency $\mathbf{A} \in [0, 1]^{B \times N \times N}$:

$$\mathbf{H}^{(t)} = \sigma \left(\mathbf{H}^{(t-1)} \mathbf{W}_1^{(t)} + \mathbf{A} \mathbf{H}^{(t-1)} \mathbf{W}_2^{(t)} \right), \quad (25)$$

where the batch dimension is broadcast. The matrix product $\mathbf{A} \mathbf{H}^{(t-1)}$ replaces `scatter_add`. Each entry $\tilde{A}_{ij}$ participates directly in the multiplication, so autograd can compute

$\partial \mathbf{H}^{(t)} / \partial \tilde{A}_{ij}$. The weights $\mathbf{W}_1^{(t)}, \mathbf{W}_2^{(t)}$ are copied from the pretrained sparse model and kept frozen. This step is taken directly from GIN-Graph [Sun et al.(2025)] and similar dense rewrites for differentiable graph generation. It is stated here to contrast with the 2-GNN case.

3.5.5 Dense 2-GNN

The same sparse-to-dense logic is applied to the 2-GNN branch. The construction is more involved than for the 1-GNN because the 2-set neighbourhood depends on adjacency lookups, which must also be made continuous.

Recall from Section 3.2 that the 2-GNN operates on node pairs $\{i, j\}$, and the neighbourhood definition (Equation 9) requires checking whether a candidate node w is adjacent in the original graph: $A_{iw} = 1$ when replacing i , or $A_{jw} = 1$ when replacing j . In the sparse implementation, this adjacency check uses a CSR (compressed sparse row) lookup: given node i , iterate over its edge list to find all w with $(i, w) \in E$. This is fast for discrete graphs, but it is the same type of discrete operation that blocks gradients, since the neighbour set is a list of integer indices with no derivative connecting it to $\tilde{A}$.

In the dense rewrite, this binary check becomes a continuous weight. Instead of checking whether w is a neighbour of i (answer: 0 or 1), we use $\tilde{A}_{iw} \in [0, 1]$ directly. Candidate pair features $\mathbf{g}[j, w]$ are multiplied by $\tilde{A}_{iw}$, contributing to the aggregation in proportion to $\tilde{A}_{iw}$. When $\tilde{A}_{iw} \approx 0$, the contribution is zero; when $\tilde{A}_{iw} \approx 1$, it is full; intermediate values contribute proportionally. On discrete $\{0, 1\}$ inputs, this reproduces the original neighbourhood rule exactly.

First, we construct pair features from the 1-GNN node embeddings $\mathbf{H}^{(T_1)} \in \mathbb{R}^{B \times N \times d_T}$. For each pair (i, j) , we concatenate the two node embeddings in canonical min-max order with the continuous adjacency value $\tilde{A}_{ij}$:

$$\mathbf{F}[i, j] = [\mathbf{h}_{\min(i, j)} \parallel \mathbf{h}_{\max(i, j)} \parallel A_{ij}] \in \mathbb{R}^{2d_T+1}, \quad (26)$$

The canonical ordering (min, max) ensures $\mathbf{F}[i, j] = \mathbf{F}[j, i]$, matching the unordered pair semantics of Equation (8). The last entry A_{ij} is the isomorphism type indicator. In the sparse model this was a hard bit $\mathbb{I}[(i, j) \in E] \in \{0, 1\}$; here it is the continuous value from the generator.

For pair-level message passing, recall from Equation (9) that each 2-set $\{i, j\}$ has two kinds of neighbours: pairs formed by replacing i with some w adjacent to i , and pairs formed by replacing j with some w adjacent to j . In the dense version, “adjacent to i ” is the continuous weight $\tilde{A}_{iw}$. Using the transformed features $\mathbf{g} = \mathbf{F} \mathbf{W}_4^{(\ell)} \in \mathbb{R}^{B \times N \times N \times d_T}$, the aggregation for pair (i, j) sums over all candidate w , weighted by $\tilde{A}$:

$$\text{agg}_1[i, j] = \sum_{\substack{w=1 \\ w \neq j}}^N A_{iw} \cdot \mathbf{g}[j, w] \quad (\text{replace } i \text{ with } w \text{ adjacent to } i), \quad (27)$$

$$\text{agg}_2[i, j] = \sum_{\substack{w=1 \\ w \neq i}}^N A_{jw} \cdot \mathbf{g}[i, w] \quad (\text{replace } j \text{ with } w \text{ adjacent to } j). \quad (28)$$

The exclusions $w \neq j$ in agg_1 and $w \neq i$ in agg_2 prevent including the self-pair $\{j, j\}$ or $\{i, i\}$, which is not a valid 2-set. The self-loop exclusion ($A_{ii} = 0$) removes $w = i$

from agg_1 and $w = j$ from agg_2 , but the terms $w = j$ in agg_1 and $w = i$ in agg_2 must be explicitly masked. We zero the diagonal of $\mathbf{g}$ (setting $\mathbf{g}[k, k] = \mathbf{0}$ for all k) before summation, which eliminates both boundary terms: $\mathbf{g}[j, j] = \mathbf{0}$ removes the $w = j$ term in agg_1 , and $\mathbf{g}[i, i] = \mathbf{0}$ removes the $w = i$ term in agg_2 .

These sums are computed via Einstein summation (`einsum`) over the masked $\mathbf{g}$. The full 2-GNN layer update is:

$$\mathbf{F}^{(\ell+1)} = \sigma\left(\mathbf{F}^{(\ell)} \mathbf{W}_3^{(\ell)} + \text{agg}_1^{(\ell)} + \text{agg}_2^{(\ell)}\right). \quad (29)$$

After $T_2 = 2$ layers, the 2-GNN readout sums over the upper-triangular entries (one entry per canonical pair):

$$\mathbf{h}_G^{(2)} = \sum_{i < j} \mathbf{F}^{(T_2)}[i, j]. \quad (30)$$

3.5.6 Summary

The dense wrapper preserves the exact mathematical behaviour of the pretrained classifier on discrete graphs while extending its domain to continuous adjacency matrices. It is an implementation change, not a model change: all pretrained weights are reused unchanged. For the 1-GNN branch, the change is replacing `scatter_add` with `matmul`, as in GIN-Graph [Sun et al.(2025)]. For the 2-GNN branch, the discrete adjacency check $A_{iw} \in \{0, 1\}$ is replaced by a continuous weight $\tilde{A}_{iw} \in [0, 1]$ that participates in the computation graph. Without this, the GIN-Graph training loop, which requires $\partial f_\Phi / \partial \tilde{A}$ to be well-defined, cannot be run against a 1-2-GNN classifier.

3.6 Validation Score

At evaluation time, both the generator and the classifier are frozen; no gradients are computed. We sample 100 graphs per class at temperature $\tau = 0.1$ (which produces sharper, more discrete outputs than the training temperature of $\tau = 1.0$) and assign each graph a score that quantifies its quality as an explanation. The score must be low if the graph fails on any of three axes (class identity, internal representation, or structural plausibility), so that success on one axis cannot compensate for failure on another.

The composite validation score [Sun et al.(2025)] is:

$$v = (s \cdot p \cdot d)^{1/3}, \quad (31)$$

where p measures the classifier’s prediction, s measures where the graph sits in the classifier’s internal embedding space, and d measures whether the graph has a realistic amount of connectivity.

The geometric mean enforces non-compensation between components. If a generated graph has $p = 0.95$, $s = 0.95$, but $d = 0.05$ (correct class, correct embedding region, wrong connectivity), the geometric mean gives $v \approx 0.36$, correctly indicating failure. An arithmetic mean would give $v \approx 0.65$, above the validity threshold, hiding the structural problem. We now define each component.

3.6.1 Prediction Probability (p)

The prediction probability is the frozen classifier’s softmax output for the target class:

$$p = p_{\Phi}(y = c \mid \tilde{G}).$$

This is the same quantity the generator was trained to maximise via $\mathcal{L}_{\text{GNN}}$ (Equation 22). For a well-trained generator, p is close to 1 at evaluation time. This is expected: the generator spent hundreds of epochs optimising this quantity. The component p therefore functions as a sanity check that the generator converged, not as a discriminator between good and poor generators. A generator producing graphs with $p < 0.5$ did not converge.

3.6.2 Embedding Similarity (s)

For any graph G the classifier processes, the forward pass produces a single vector $\mathbf{h}_G$ representing the entire graph. In the 1-GNN, this vector is the sum pool of the final node embeddings: $\mathbf{h}_G = \sum_{v \in V} \mathbf{h}_v^{(T_1)} \in \mathbb{R}^{d_T}$ (Equation 6). In the 1-2-GNN, it is the concatenation of the node-level and pair-level readouts: $\mathbf{h}_G = [\mathbf{h}_G^{(1)} \parallel \mathbf{h}_G^{(2)}] \in \mathbb{R}^{2d_T}$ (Equation 11). One vector per graph, in the classifier’s embedding space. Every forward pass on any graph produces exactly one such vector.

The *class centroid* $\boldsymbol{\mu}_c$ is the mean of $\mathbf{h}_G$ over all real training graphs in class c :

$$\boldsymbol{\mu}_c = \frac{1}{|\mathcal{G}_c|} \sum_{G \in \mathcal{G}_c} \mathbf{h}_G.$$

For MUTAG class 0, $\mathcal{G}_c$ contains 125 molecules, so $\boldsymbol{\mu}_c$ is the average of 125 embedding vectors. The centroid sits in the same space as any individual $\mathbf{h}_G$ and marks the average location where real class- c graphs lie according to the classifier. It is computed once using the classifier’s sparse implementation (real graphs are discrete, so sparse is appropriate), stored in the GIN-Graph checkpoint, and reused for every evaluation.

The embedding similarity is then the cosine similarity between the generated graph’s embedding and this centroid:

$$s = \cos(\mathbf{h}_{\tilde{G}}, \boldsymbol{\mu}_c) = \frac{\mathbf{h}_{\tilde{G}} \cdot \boldsymbol{\mu}_c}{\|\mathbf{h}_{\tilde{G}}\| \|\boldsymbol{\mu}_c\|}, \quad \text{where } \boldsymbol{\mu}_c = \frac{1}{|\mathcal{G}_c|} \sum_{G \in \mathcal{G}_c} \mathbf{h}_G. \quad (32)$$

Cosine similarity measures the angle between two vectors, normalised so that identical direction gives $s = 1$ and orthogonal vectors give $s = 0$. A high s means the generated graph occupies the same region of embedding space as real class- c graphs. A low s means the classifier assigns the graph to class c but via an internal representation far from typical class members; the generator has found an input that triggers the correct output without resembling the real data in the classifier’s internal representation.

Note on the computation of s . The GIN-Graph reference implementation [Sun et al.(2025)] uses the prediction probability p as a proxy for embedding similarity, setting $s = p$ and reducing the validation score to $v = (p^2 \cdot d)^{1/3}$. In this work, s is computed as the actual cosine similarity between $\mathbf{h}_{\tilde{G}}$ and $\boldsymbol{\mu}_c$, using the dense wrapper’s `get_embedding()` method to extract the embedding from the frozen classifier for a continuous-valued generated graph. On PROTEINS Enzyme, this distinction is visible in the results: both models achieve high p (0.919 and 0.998) but low s (0.356 and 0.151), a difference that would not appear under the $s = p$ proxy.

3.6.3 Degree Score (d)

The average degree of a graph is $\bar{d}_{\tilde{G}} = 2|\tilde{E}|/|\tilde{V}|$: total degree divided by node count, since each edge contributes 1 to each of its two endpoints. Let μ_d and σ_d be the mean and standard deviation of average degrees across real class- c graphs, both precomputed once from the training set. The degree score is:

$$d = \exp\left(-\frac{(\bar{d}_{\tilde{G}} - \mu_d)^2}{2\sigma_d^2}\right), \quad (33)$$

which is a Gaussian centred at the class mean μ_d with width σ_d . When $\bar{d}_{\tilde{G}} = \mu_d$, the score is $d = 1$. As the generated degree deviates from the mean, d decreases: one standard deviation off gives $d \approx 0.61$, two standard deviations give $d \approx 0.14$, three give $d \approx 0.01$. Classes with high variance in graph sizes get a wider (more permissive) Gaussian than classes with uniform sizes.

The degree score provides a structural sanity check that s and p cannot provide. Both s and p operate in the classifier’s learned embedding space. A generator could produce graphs with correct class identity and correct embedding location but with edge counts far from any real graph. The degree score catches this failure mode. It does not capture fine-grained structural properties (motif frequencies, degree distributions), but it reliably rejects graphs with wrong connectivity at the coarsest level.

3.6.4 Validity Threshold

A generated graph is considered **valid** if two conditions hold: (i) $v > 0.5$, and (ii) $\bar{d}_{\tilde{G}}$ falls within 3 standard deviations of μ_d . The degree score d therefore appears twice: once as a soft component inside v , and once as a hard gate via the 3σ condition. The hard gate catches graphs with abnormal connectivity regardless of their s and p values. A graph with $\bar{d}_{\tilde{G}}$ far outside the class range is rejected even if it scores well on the other two components. The validity rates reported in Section 5 are filtered on both conditions.

3.6.5 Granularity (κ)

We additionally report:

$$\kappa = 1 - \min\left(1, \frac{n_{\tilde{G}}}{\bar{n}_c}\right), \quad (34)$$

where $n_{\tilde{G}}$ is the number of active (non-isolated) nodes in the generated graph and $\bar{n}_c$ is the mean node count of real class- c graphs (the same $\bar{n}_c$ used in $\mathcal{L}_{\text{size}}$, Equation 24). When $n_{\tilde{G}} \geq \bar{n}_c$, the ratio saturates at 1 and $\kappa = 0$: the explanation is graph-sized. When $n_{\tilde{G}} \ll \bar{n}_c$, κ approaches 1: the explanation is a small substructure.

In these experiments, $\kappa \approx 0$ for all configurations. This is a structural property of GIN-Graph, not a failure. The generator outputs fixed-size $N \times N$ tensors and $\mathcal{L}_{\text{size}}$ pushes $\hat{n}$ toward $\bar{n}_c$, so explanations are graph-sized by construction. Model-level explainers answer “what does a typical class member look like?” rather than “which subgraph drives this prediction?”; the latter requires instance-level methods such as GNNExplainer. We report κ for completeness and as a diagnostic; this limitation is discussed further in Section 6.

3.6.6 Evaluation Protocol

Each configuration generates 100 graphs and reports: (i) the validity rate, the fraction passing both the $v > 0.5$ and the 3σ degree conditions; (ii) the mean validation score, averaged across all 100 graphs; (iii) the component decomposition s, p, d (Table 6), where patterns such as high p with low s on PROTEINS Enzyme would not appear under the $s = p$ proxy; and (iv) the top-ranked explanations sorted by v , shown in the figures. All four quantities are forward-pass computations on frozen networks. No training or gradients are involved.

4 Experimental Setup

4.1 Datasets

We evaluate on two standard graph classification benchmarks (Table 1).

Table 1: Dataset statistics. GIN Gen is the maximum number of nodes used for explanation generation.

Dataset	Graphs	Node Feats	Max Nodes	GIN Gen	Task
MUTAG	188	7 (atom types)	28	28	Mutagenicity
PROTEINS	1113	3 (sec. struct.)	620	50	Enzyme vs. non-enzyme

MUTAG [Debnath et al.(1991)] contains 188 molecular graphs labelled by mutagenic effect on *Salmonella typhimurium*. Nodes represent atoms (C, N, O, F, I, Cl, Br) and edges represent chemical bonds. The two classes are *Mutagen* (class 0, 125 graphs) and *Non-Mutagen* (class 1, 63 graphs).

PROTEINS [Borgwardt et al.(2005)] contains 1113 protein graphs where nodes are secondary structure elements (helix, sheet, coil/turn) connected if they are neighbours in the amino acid sequence or within a distance threshold in 3D space. The classes are *Non-Enzyme* (class 0) and *Enzyme* (class 1). For GIN-Graph generation, we cap the graph size at 50 nodes (the median protein graph has ~ 26 nodes), as the explanations highlight key substructures rather than reproducing entire large graphs.

4.2 Model Configuration

All k -GNN models use a hidden dimension of $d_T = 64$. The 1-GNN uses $T_1 = 3$ message-passing layers; the 1-2-GNN uses 3 layers for the 1-GNN component and $T_2 = 2$ layers for the 2-GNN component. Both use dropout of 0.5 and are trained with Adam (lr = 0.01) for 100 epochs with cross-entropy loss. Data is split 80/20 into train/test with a fixed seed of 42.

The GIN-Graph generator uses a latent dimension $d_z = 32$, hidden dimension $d_h = 128$, and is trained for 300 epochs with Adam (lr = 0.001) and batch size 64. WGAN-GP uses $\lambda_{GP} = 10$ and trains the discriminator once per generator update ($n_{critic} = 1$). The dynamic weighting uses $p = 0.4$, $k = 10$, $\lambda_{min} = 0$, $\lambda_{max} = 1$.

Computational cost. Table 2 reports wall-clock times on CPU. The 2-GNN component incurs substantial overhead due to the $O(n^2)$ pair construction and message passing. On PROTEINS, the 1-2-GNN is $\sim 326\times$ slower per forward pass than the 1-GNN, reflecting the cost of processing all $\binom{n}{2}$ node pairs for graphs with up to 620 nodes. GIN-Graph generation uses the dense wrapper, where the 1-2-GNN overhead is $\sim 36\text{--}74\times$ at the fixed generation size ($N = 28$ for MUTAG, $N = 50$ for PROTEINS).

Table 2: Wall-clock inference times on CPU. k -GNN: sparse forward pass (batch of 64 graphs). GIN-Graph: dense wrapper generation (100 graphs).

Dataset	Model	k -GNN (ms/batch)	GIN-Graph gen (s/100)
MUTAG	1-GNN	1.9	0.02
MUTAG	1-2-GNN	99.5 (52 $\times$)	1.48 (74 $\times$)
PROTEINS	1-GNN	6.8	0.04
PROTEINS	1-2-GNN	2214 (326 $\times$)	1.43 (36 $\times$)

4.3 Evaluation Protocol

For each model-dataset-class combination, we generate 100 explanation graphs at evaluation temperature $\tau = 0.1$ and evaluate them using the validation score (Equation 31). We report:

- **Validity rate:** fraction of explanations passing degree and score thresholds;
- **Mean validation score:** averaged over all 100 generated graphs;
- **Component scores:** embedding similarity (s), prediction probability (p), degree score (d);
- **Granularity:** how fine-grained the explanations are.

5 Results

5.1 k -GNN Classification Performance

Table 3 summarises the classification results. On MUTAG, both models achieve identical best test accuracy of 89.5%, indicating that the additional pairwise message passing does not provide a classification advantage on this small molecular dataset. On PROTEINS, both models achieve comparable classification accuracy (79.8% vs. 78.9%) after 100 epochs of training, with the 1-2-GNN now processing all $\binom{n}{2}$ node pairs via a numba-accelerated kernel instead of sampling a subset. The 1-2-GNN’s best accuracy occurs at epoch 75, showing that the model benefits from extended training when given access to the full pairwise structure.

5.2 GIN-Graph Explanation Quality: MUTAG

Table 4 presents the GIN-Graph results on MUTAG. All configurations were fully trained for 300 epochs, enabling fair cross-model comparison on both classes.

Table 3: k -GNN classification results. Best test accuracy reported over all epochs.

Dataset	Model	Params	Final Test Acc	Best Acc	Best Epoch
MUTAG	1-GNN	21,570	81.6%	89.5%	84
MUTAG	1-2-GNN	50,370	76.3%	89.5%	18
PROTEINS	1-GNN	21,058	75.8%	79.8%	120
PROTEINS	1-2-GNN	49,858	73.1%	78.9%	75

Table 4: GIN-Graph explanation quality on MUTAG. All models trained for 300 epochs.

Class	Model	Valid%	Val Score	s	p	d
0 (Mutagen)	1-GNN	32%	0.320	0.700	1.000	0.210
0 (Mutagen)	1-2-GNN	41%	0.434	0.771	1.000	0.314
1 (Non-Mutagen)	1-GNN	78%	0.681	0.935	1.000	0.490
1 (Non-Mutagen)	1-2-GNN	82%	0.735	0.933	1.000	0.565

Both models achieve successful generation on both classes, with the 1-2-GNN showing somewhat higher validity rates (82% vs. 78% on class 1, 41% vs. 32% on class 0) and validation scores (0.735 vs. 0.681 on class 1). However, the more significant finding is not the quantitative difference—which may not be statistically significant given the single-seed setup (Section ??)—but the *qualitative divergence* in what the two models generate.

Despite identical classification accuracy (89.5%), the two models produce strikingly different class-0 explanation families (Figure 1). Across the saved top-ranked explanation set, the 1-GNN remains C/N/O-dominated (42.9% C, 13.9% N, 13.2% O), while the 1-2-GNN shifts toward a much more halogen-heavy pattern (31.1% F, 1.1% Br) with almost no nitrogen (0.4% N). The population-level comparison in Section 5.6 shows the same pattern: on MUTAG class 0, the 1-2-GNN generator overproduces halogens much more strongly than the 1-GNN. This divergence reveals that the two architectures have learned fundamentally different internal representations of the same class.

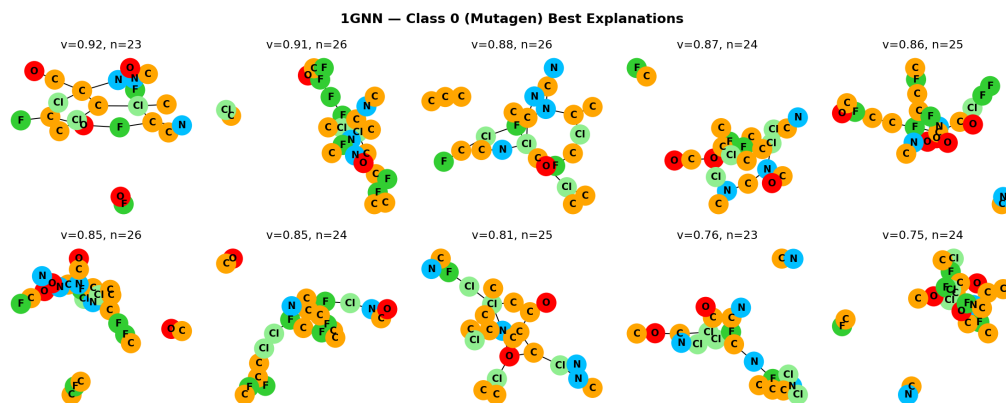
On class 0 (Mutagen), both models struggle with low validity rates despite Mutagen being the majority class ($125/188 = 66.5\%$). This likely stems from structural diversity: mutagenic compounds exhibit varied functional groups—nitro, amino, halogen—making a single representative graph harder to produce than the more structurally uniform Non-Mutagen class. Both models achieve perfect prediction probability ($p = 1.0$).

Figure 2 shows the top-ranked explanation graphs for both models on MUTAG class 1.

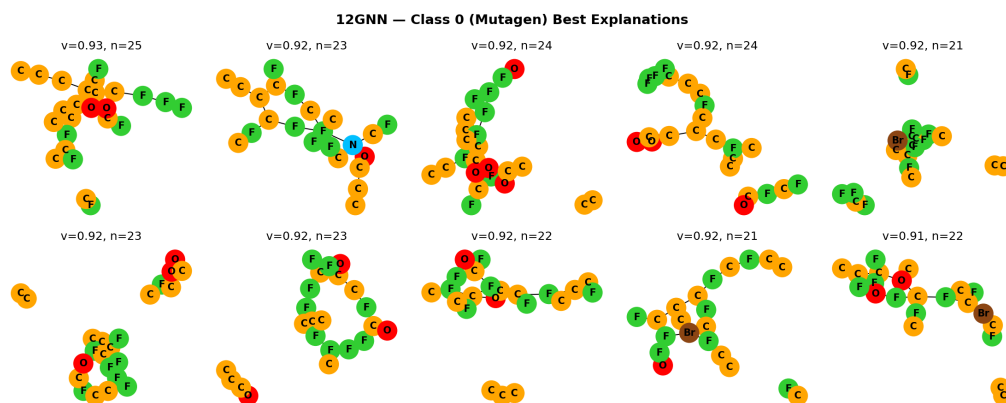
The top 1-2-GNN explanations achieve near-perfect validation scores (0.986), with consistent graph sizes (24–27 nodes, 27–30 edges). The 1-GNN explanations achieve similarly high scores (top score 0.984), with comparable structural variation.

5.3 GIN-Graph Explanation Quality: PROTEINS

Table 5 presents results on PROTEINS. All configurations were fully trained for 300 epochs with graph size and node type regularisation (see Section 3.4). Unlike MUTAG, the two models show complementary strengths, with each achieving higher scores on a

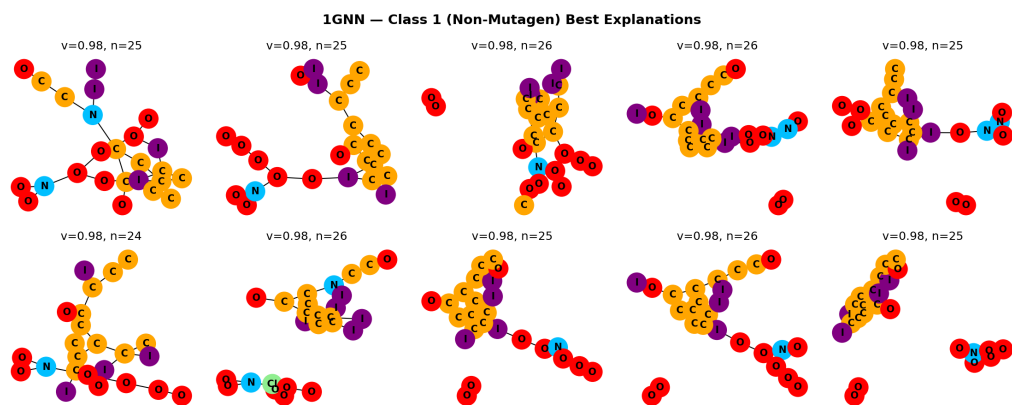


(a) 1-GNN explanations for Mutagen

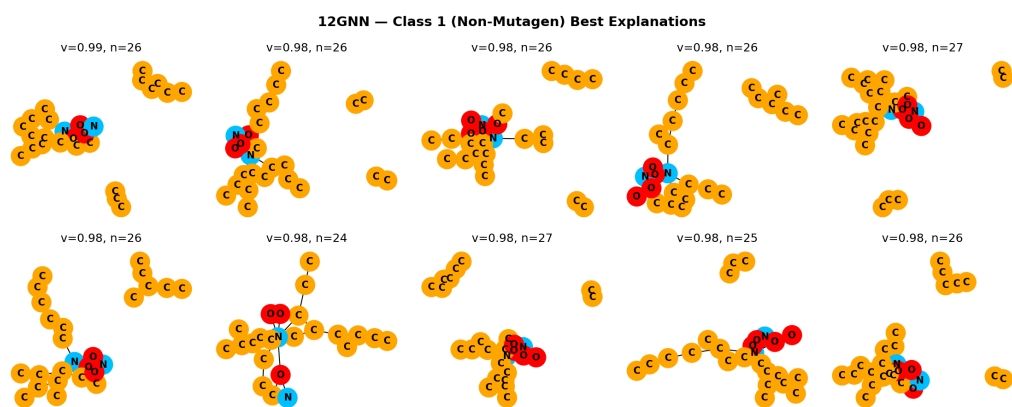


(b) 1-2-GNN explanations for Mutagen

Figure 1: Top-ranked GIN-Graph explanations on MUTAG class 0 (Mutagen). The 1-GNN produces mostly C/N/O-rich structures, while the 1-2-GNN produces much more halogen-heavy structures. Node colours indicate atom types (see Figure 3).



(a) 1-GNN explanations for Non-Mutagen



(b) 1-2-GNN explanations for Non-Mutagen

Figure 2: Top-ranked GIN-Graph explanations on MUTAG class 1 (Non-Mutagen). Node colours indicate atom types (see Figure 3).

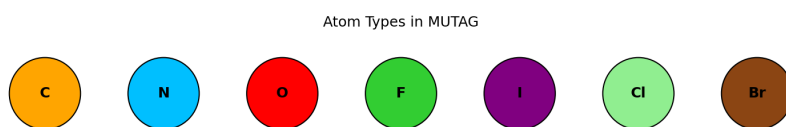


Figure 3: MUTAG atom type colour legend. Atoms: C (orange), N (blue), O (red), F (green), I (purple), Cl (light green), Br (brown).

different class.

Table 5: GIN-Graph explanation quality on PROTEINS. All models trained for 300 epochs with structural regularisation.

Class	Model	Valid%	Val Score	s	p	d
0 (Non-Enzyme)	1-GNN	100%	0.986	0.987	0.986	0.984
0 (Non-Enzyme)	1-2-GNN	100%	0.760	0.782	0.564	0.996
1 (Enzyme)	1-GNN	37%	0.330	0.356	0.919	0.837
1 (Enzyme)	1-2-GNN	94%	0.524	0.151	0.998	0.961

The 1-GNN achieves near-perfect scores on class 0 (Non-Enzyme) across all components ($v = 0.986$, $s = 0.987$, $p = 0.986$), while the 1-2-GNN achieves lower but still functional prediction probability ($p = 0.564$) and overall validation score ($v = 0.760$). On class 1 (Enzyme), both models achieve high prediction probability ($p = 0.919$ and 0.998), but the 1-GNN struggles with validity (37%) while the 1-2-GNN achieves 94% validity. Notably, both models show low embedding similarity on Enzyme ($s = 0.356$ and 0.151), indicating that generated Enzyme graphs remain distant from the real embedding centroid despite being correctly classified.

Both models achieve comparable classification accuracy (79.8% vs. 78.9%), so this complementary pattern reflects genuine differences in what the two architectures learn about protein structure rather than a simple classifier quality gap. The 1-GNN’s node-level representations capture Non-Enzyme patterns more effectively, while the 1-2-GNN’s pairwise representations enable higher-validity Enzyme generation.

The class-specific strengths are visually evident in the explanation figures. The 1-GNN class 0 explanations (Figure 4) show high-quality graphs with validation scores $v \approx 0.99$, while its class 1 explanations have 37% validity, with the valid graphs showing lower embedding similarity. The 1-2-GNN explanations (Figure 5) achieve high validity on both classes (100% class 0, 94% class 1), with moderate prediction probability on class 0 ($p = 0.564$) and low embedding similarity on class 1 ($s = 0.151$). This complementary pattern—each model generating higher-quality explanations for a different class—suggests that the node-level and pairwise representations capture different aspects of protein structure.

5.4 Metric Decomposition

Table 6 provides a detailed comparison of the three validation score components across all configurations, now with all models trained for 300 epochs.

On MUTAG, both models achieve perfect prediction probability ($p = 1.0$) on both classes, indicating that both classifiers provide strong gradient signal to the generators. The degree score shows the largest variation: the 1-2-GNN produces graphs with more realistic average degree on both classes (0.565 vs. 0.490 on class 1, 0.314 vs. 0.210 on class 0). Embedding similarity is comparable on class 1 ($s \approx 0.93$) and somewhat higher for the 1-2-GNN on class 0 (0.771 vs. 0.700).

On PROTEINS, the metric profiles reveal how each architecture interacts differently with the generator. The 1-GNN achieves near-perfect scores on class 0 ($s = 0.987$, $p = 0.986$,

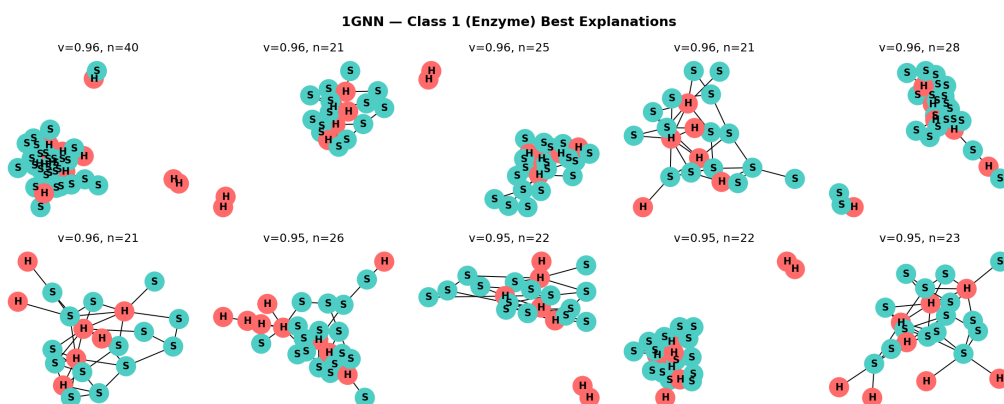
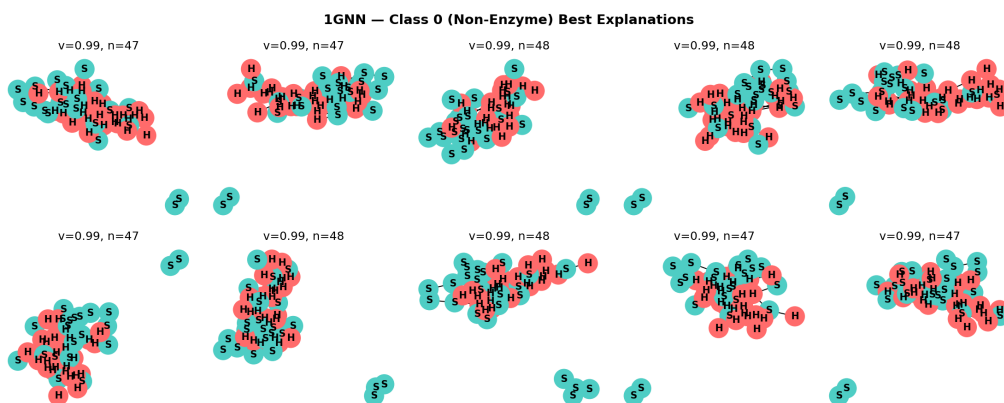
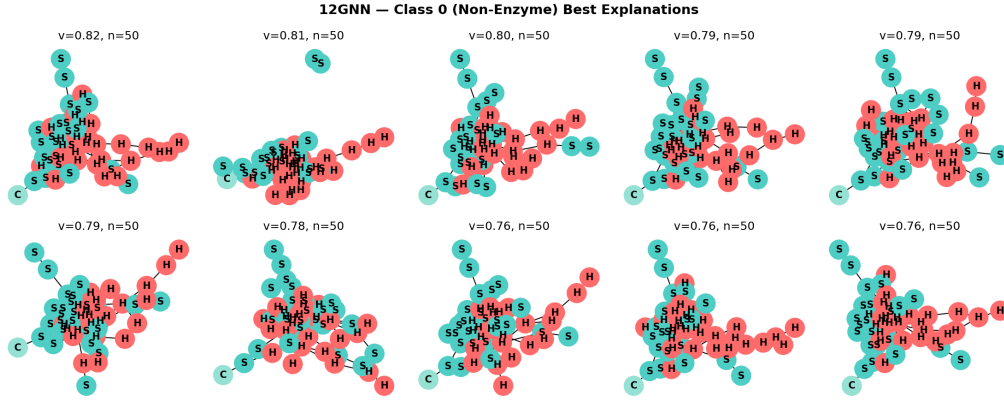


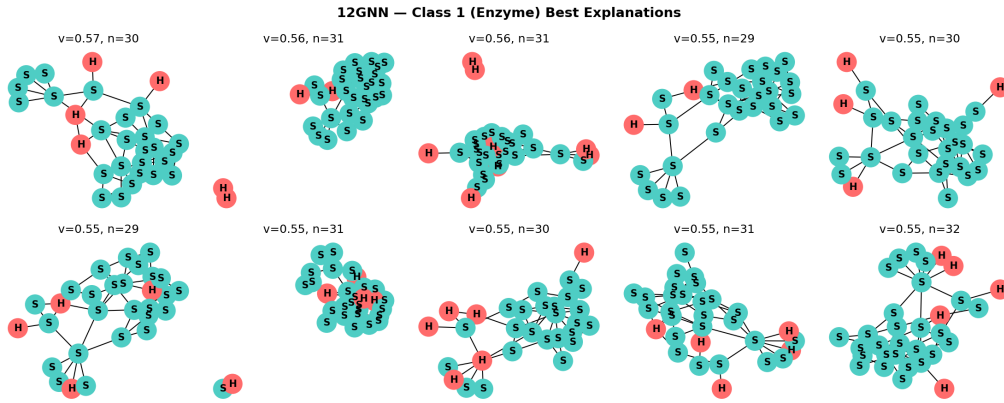
Figure 4: Top-ranked GIN-Graph explanations on PROTEINS (1-GNN). Node colours indicate secondary structure types: helix (H, pink), sheet (S, teal), coil/turn (C, green).

Table 6: Validation score decomposition for all GIN-Graph models (300 epochs).

Dataset	Class	Model	s (emb. sim)	p (pred. prob)	d (degree)
MUTAG	0 (Mutagen)	1-GNN	0.700	1.000	0.210
MUTAG	0 (Mutagen)	1-2-GNN	0.771	1.000	0.314
MUTAG	1 (Non-Mut.)	1-GNN	0.935	1.000	0.490
MUTAG	1 (Non-Mut.)	1-2-GNN	0.933	1.000	0.565
PROTEINS	0 (Non-Enz.)	1-GNN	0.987	0.986	0.984
PROTEINS	0 (Non-Enz.)	1-2-GNN	0.782	0.564	0.996
PROTEINS	1 (Enzyme)	1-GNN	0.356	0.919	0.837
PROTEINS	1 (Enzyme)	1-2-GNN	0.151	0.998	0.961



(a) 1-2-GNN explanations for Non-Enzyme



(b) 1-2-GNN explanations for Enzyme

Figure 5: Top-ranked GIN-Graph explanations on PROTEINS (1-2-GNN). Class 0 explanations achieve validation scores $v = 0.76$ – 0.82 with 50 nodes, while class 1 explanations achieve $v = 0.52$ – 0.58 with 30–50 nodes.

$d = 0.984$) but lower embedding similarity on class 1 ($s = 0.356$), indicating generated Enzyme graphs diverge from the real embedding centroid despite being correctly classified ($p = 0.919$). The 1-2-GNN achieves moderate prediction probability on class 0 ($p = 0.564$) with high embedding similarity ($s = 0.782$) and near-perfect degree score ($d = 0.996$), while on class 1 it achieves near-perfect prediction probability ($p = 0.998$) but very low embedding similarity ($s = 0.151$). The degree scores are generally high across PROTEINS ($d > 0.83$), confirming that structural realism is maintained by the WGAN-GP component.

5.5 Training Dynamics

Figure 6 shows a representative training curve from the MUTAG 1-2-GNN class 1 experiment. The three loss curves illustrate the interplay between the WGAN-GP adversarial training and the GNN guidance objective over 300 epochs.

During the first ~ 120 epochs ($\lambda \approx 0$), the generator loss and discriminator loss dominate: the generator learns to produce structurally realistic graphs that fool the discriminator, without pressure to target a specific class. As the dynamic weighting $\lambda(t)$ increases (Equation 23), the GNN guidance loss takes over, steering the generator toward class-specific patterns. This transition is visible as the GNN loss drops sharply while the adversarial losses plateau.

The training behaviour varies across configurations. On MUTAG, all four generators converge smoothly, consistent with the small graph sizes ($N = 28$) and low feature dimensionality (7 atom types). On PROTEINS, the training dynamics are more complex: the 1-2-GNN class 0 generator achieves moderate prediction probability ($p = 0.564$ at evaluation), lower than the 1-GNN’s near-perfect alignment ($p = 0.986$) on the same class. This gap reflects the difficulty of optimising through the 1-2-GNN’s higher-dimensional embedding space ($\mathbb{R}^{128}$ vs. $\mathbb{R}^{64}$), where the generator must satisfy both node-level and pair-level classifier components simultaneously.

5.6 Generated Dataset Comparison

To evaluate the fidelity and cross-model transferability of generated explanations at scale, we generate 500 graphs per class from each GIN-Graph generator and compare the resulting synthetic datasets against the real data. This population-level analysis complements the per-graph metrics reported in Sections 5.2–5.4 by revealing systematic differences between what each model has learned. No retraining is performed—all graphs are generated from the existing checkpoints.

5.6.1 Structural Fidelity

Table 7 summarises the structural comparison. Across both datasets, degree distributions are matched more reliably than graph-size distributions. Kolmogorov–Smirnov statistics on degree range from 0.09 to 0.21, with generated mean degrees remaining close to the real values in every configuration. By contrast, the size KS statistics are substantially larger, indicating that the generators preserve local connectivity more successfully than the global population shape.

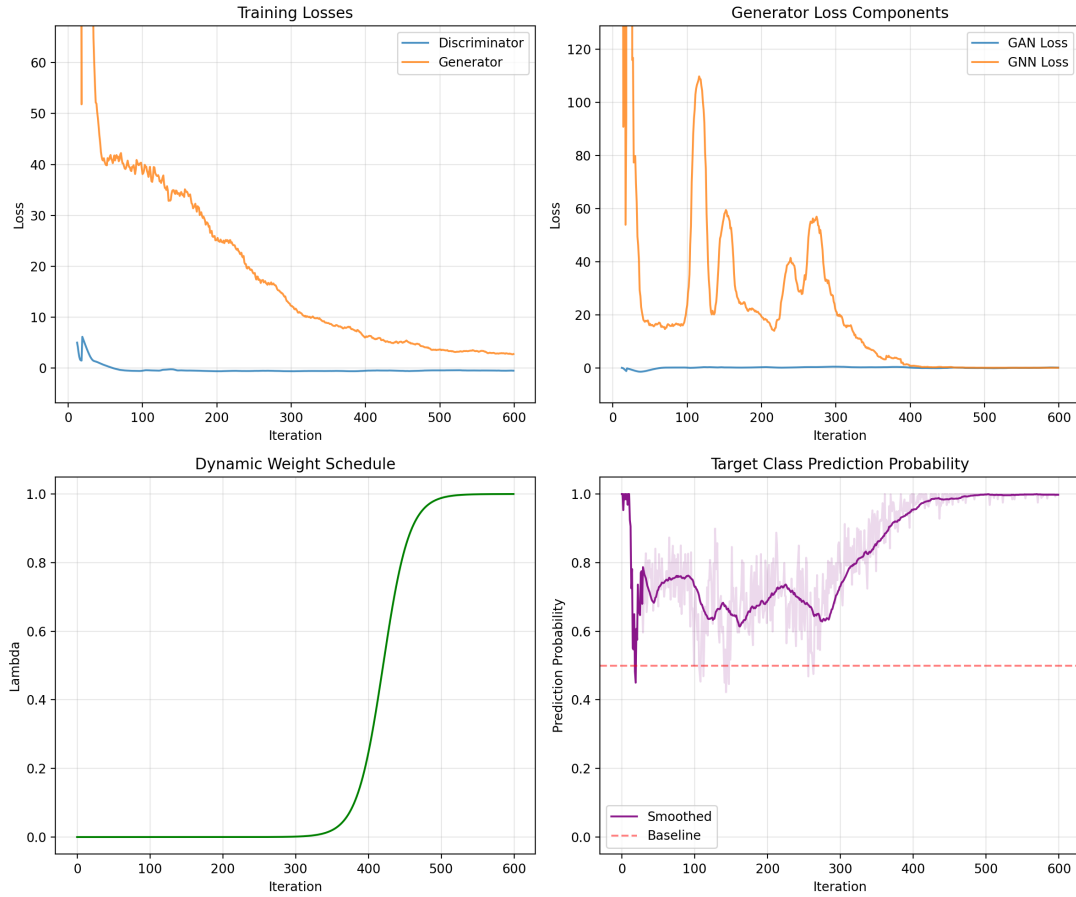


Figure 6: Training curves for GIN-Graph on MUTAG 1-2-GNN class 1, showing generator loss, discriminator loss, and GNN guidance loss over 300 epochs. The dynamic weighting schedule shifts emphasis from adversarial realism (early epochs) to class-specific GNN guidance (later epochs).

Size regularisation helps on PROTEINS, but only partially. On class 0 (Non-Enzyme), generated graphs average 46.6 nodes (1-GNN) and 50.0 nodes (1-2-GNN) versus a real mean of 48.7. On class 1 (Enzyme), the 1-GNN matches the real mean size closely (24.0 vs. 23.7), whereas the 1-2-GNN remains oversized (29.5 vs. 23.7). On MUTAG ($N = 28$), all generators still produce oversized graphs: the generated means lie between 23 and 27 nodes, while the real means lie between 14 and 20. This reflects the fixed generator size and the tendency of the Gumbel-Softmax sampling to keep most nodes active.

Node type distributions reveal a mixed rather than uniformly positive picture. On MUTAG class 1 (Non-Mutagen), the 1-2-GNN is the clearest success case: it generates C/O-dominated graphs with 80.0% C and 17.1% O, much closer to the real class distribution than the 1-GNN, which introduces atypical iodine (7.7%) and fluorine (2.0%). On MUTAG class 0 (Mutagen), however, the 1-2-GNN over-emphasises halogens (26.7% F, 11.0% Br, 3.6% I) and underproduces N, whereas the 1-GNN stays closer to the real C/N/O proportions. On PROTEINS class 0, the 1-2-GNN closely matches the real secondary-structure distribution (H: 50.9% vs. 50.4%, S: 47.1% vs. 46.5%, C: 2.0% vs. 3.1%). On PROTEINS class 1, by contrast, both models overproduce sheets and underproduce helix/coil, with the 1-2-GNN class 1 generator being the least faithful (88.9% S vs. 56.2% real).

Table 7: Structural comparison: 500 generated vs. real graphs per configuration. $D_{\text{KS}}^{\text{deg}}$ and $D_{\text{KS}}^{\text{size}}$ denote Kolmogorov–Smirnov statistics on degree and graph-size distributions, respectively. $\bar{d}$ = mean degree, $\bar{n}$ = mean graph size.

Configuration	$D_{\text{KS}}^{\text{deg}} \downarrow$	$D_{\text{KS}}^{\text{size}} \downarrow$	$\bar{d}_{\text{real}} / \bar{d}_{\text{gen}}$	$\bar{n}_{\text{real}} / \bar{n}_{\text{gen}}$
<i>MUTAG</i>				
1-GNN class 0	0.120	0.891	2.09 / 2.06	13.9 / 23.8
1-GNN class 1	0.094	0.893	2.24 / 2.23	19.8 / 27.3
1-2-GNN class 0	0.125	0.881	2.09 / 2.05	13.9 / 23.1
1-2-GNN class 1	0.155	0.594	2.24 / 2.23	19.8 / 24.3
<i>PROTEINS</i>				
1-GNN class 0	0.149	0.660	3.80 / 3.81	48.7 / 46.6
1-GNN class 1	0.213	0.537	3.64 / 3.73	23.7 / 24.0
1-2-GNN class 0	0.178	0.704	3.80 / 3.81	48.7 / 50.0
1-2-GNN class 1	0.165	0.738	3.64 / 3.64	23.7 / 29.5

5.6.2 Cross-Model Classification

The most revealing analysis classifies each set of 500 generated graphs with *both* k-GNN classifiers, testing whether explanations produced by one model are recognisable by the other. Table 8 reports target class agreement rates, and Figure 7 visualises the full classification breakdown.

On MUTAG, three of four configurations show high cross-model agreement: 1-GNN class 0 at 90.4%, 1-GNN class 1 at 91.6%, and 1-2-GNN class 0 at 100%. The exception is striking: the 1-2-GNN’s Non-Mutagen graphs (class 1) achieve 100% same-model agreement but only 12.4% cross-model agreement—the 1-GNN classifies 87.6% of them as *Mutagen*. The 1-2-GNN, using pairwise structural information, has learned to associate Non-Mutagen

with structural features that are invisible to the 1-GNN’s simpler node-level decision boundary.

On PROTEINS, all four generators succeed at their target class (same-model agreement $\geq 98.4\%$), enabling a clean cross-model comparison. The most revealing asymmetry is on class 0 (Non-Enzyme): 1-GNN-generated Non-Enzyme graphs achieve 100% same-model agreement but only 9.4% cross-model agreement—the 1-2-GNN classifies 90.6% of them as Enzyme. In contrast, 1-2-GNN-generated Non-Enzyme graphs achieve 98.4% same-model agreement *and* 100% cross-model agreement—the 1-GNN also accepts them as Non-Enzyme. On class 1 (Enzyme), by contrast, both models largely agree (98–100% cross-model), suggesting that some class-defining protein patterns are visible to both architectures. Taken together, the two datasets indicate model-specific explanation families rather than a uniform superiority of one classifier over the other.

Table 8: Cross-model classification of generated graphs (500 per class). Same = target class agreement by the generating model’s classifier; Cross = agreement by the other model’s classifier.

Generated by	Target class	Same	Cross
<i>MUTAG</i>			
1-GNN	Mutagen (c0)	100%	90.4%
1-GNN	Non-Mutagen (c1)	99.8%	91.6%
1-2-GNN	Mutagen (c0)	99.8%	100%
1-2-GNN	Non-Mutagen (c1)	100%	12.4%
<i>PROTEINS</i>			
1-GNN	Non-Enzyme (c0)	100%	9.4%
1-GNN	Enzyme (c1)	99.8%	99.6%
1-2-GNN	Non-Enzyme (c0)	98.4%	100%
1-2-GNN	Enzyme (c1)	100%	98.0%

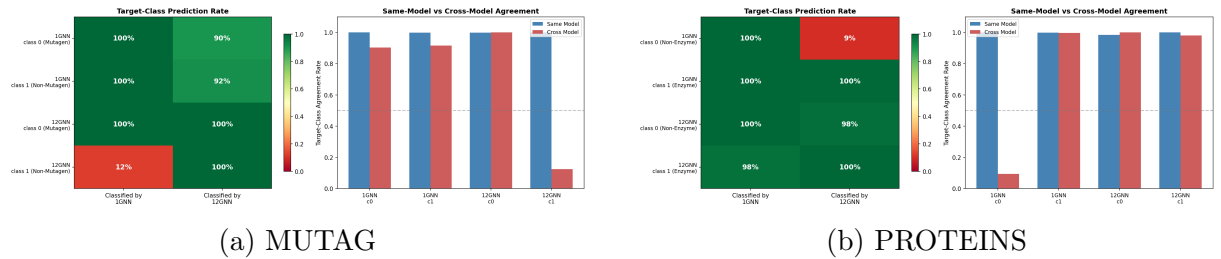


Figure 7: Cross-model classification of generated graphs. Each cell shows the fraction classified into the target class by each model. High same-model agreement (diagonal) with low cross-model agreement (off-diagonal) indicates that the two architectures have learned different decision boundaries.

5.6.3 Embedding Space Structure

Figures 8 and 9 show t-SNE projections of real and generated graph embeddings in each model’s learned representation space.

On MUTAG (Figure 8), the generated graphs are cleanly separated by class, but they do not simply fill in the real-data clouds. Instead, each model produces two large generated manifolds that sit away from the much smaller real clusters. The same-model agreement is therefore not coming from faithful sampling of the empirical distribution, but from the generator finding class-consistent regions of embedding space.

The effect is even stronger on PROTEINS (Figure 9). In the 1-GNN space, the generated Non-Enzyme and Enzyme graphs occupy detached arcs and islands that only partially overlap the real distribution. In the 1-2-GNN space, the generated graphs collapse into several very tight disconnected prototype islands. These visualisations support a more cautious interpretation of the population-level results: the generators preserve class-discriminative structure well enough to achieve high same-model agreement, but they do not reproduce the full geometry of the real dataset. The generated graphs are best interpreted as model-specific prototypes rather than faithful random samples.

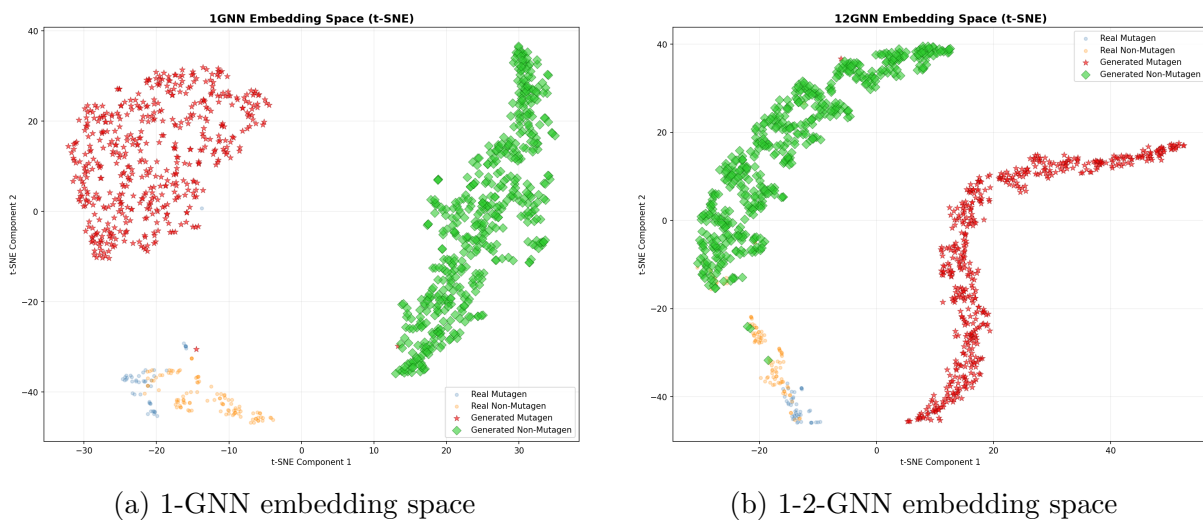


Figure 8: t-SNE projections of MUTAG real (small dots) and generated (large markers) graph embeddings. Colours distinguish Mutagen (class 0) from Non-Mutagen (class 1).

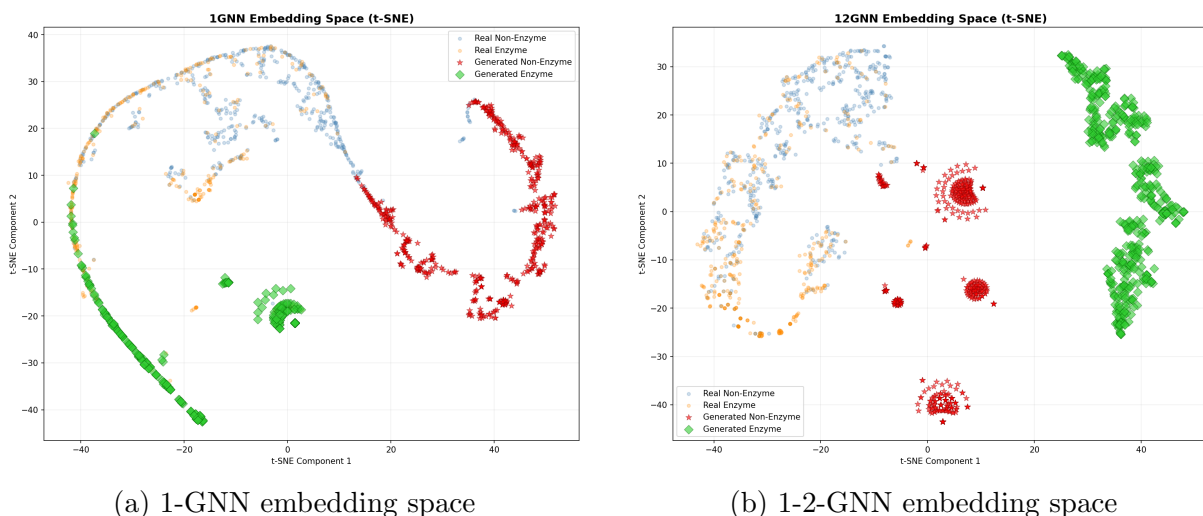


Figure 9: t-SNE projections of PROTEINS real (small dots) and generated (large markers) graph embeddings. Colours distinguish Non-Enzyme (class 0) from Enzyme (class 1).

6 Discussion

6.1 Different Architectures, Different Representations

Our central finding is that the 1-GNN and 1-2-GNN learn *qualitatively different* internal representations of graph classes, even when their classification accuracy is comparable. This is not a story of one architecture producing “better” explanations, but of two architectures encoding different structural features—differences that become visible through GIN-Graph’s model-level explanation generation.

On MUTAG, both models achieve identical classification accuracy (89.5%), yet they generate fundamentally different explanations. The 1-GNN emphasises C/N/O-rich mutagenic structures, while the 1-2-GNN emphasises much more halogen-heavy mutagenic structures. The tracked generator reports confirm that both models can produce valid explanations on both classes (32–41% validity on class 0 and 78–82% on class 1), but the *content* of those explanations differs qualitatively. This is perhaps the cleanest evidence that the two architectures learn different things: same accuracy, same dataset, different generated explanations.

On PROTEINS, the tracked generator reports show complementary strengths rather than a single dominant model. The 1-GNN achieves near-perfect scores on Non-Enzyme ($v = 0.986$), while the 1-2-GNN achieves much higher validity on Enzyme (94% vs. 37%). This complementary pattern suggests that the node-level and pairwise representations capture different aspects of protein structure.

The cross-model classification analysis (Section 5.6) provides the strongest quantitative evidence. On MUTAG, the 1-2-GNN’s Non-Mutagen graphs achieve 100% same-model agreement but only 12.4% cross-model agreement—the 1-GNN systematically misclassifies them. On PROTEINS, 1-GNN-generated Non-Enzyme graphs are rejected by the 1-2-GNN (9.4% cross-model), while 1-2-GNN-generated Non-Enzyme graphs are accepted by both models (98.4%/100%). These extreme disagreements—not marginal differences that require error bars—confirm that the two architectures have learned fundamentally different decision boundaries. The 1-2-GNN’s richer pairwise feature space enables it to make distinctions that are invisible to the 1-GNN’s node-level representations.

6.2 Qualitative Differences in Generated Explanations

Beyond the quantitative metrics, the generated explanations reveal qualitative differences in what each model has learned. On MUTAG class 0 (Mutagen), the top-ranked 1-GNN explanations are dominated by carbon, nitrogen, and oxygen (42.9% C, 13.9% N, 13.2% O), whereas the top-ranked 1-2-GNN explanations are much more halogen-heavy (31.1% F, 1.1% Br) and nearly eliminate nitrogen (0.4% N). This visual and quantitative difference is consistent with the population-level comparison, where the 1-2-GNN class-0 generator overproduces fluorine and bromine relative to the real class far more strongly than the 1-GNN. Both explanation families remain recognisably mutagenic, but the divergence demonstrates that the two architectures learn fundamentally different internal representations of the same chemical class.

On MUTAG class 1 (Non-Mutagen), both models converge on carbon-dominated backbones, but the saved top-ranked explanations again differ sharply in composition. The

1-GNN top set is split between carbon and oxygen (38.6% C, 38.2% O) and introduces substantial iodine (17.9%), whereas the 1-2-GNN top set is more conventional and concentrated on carbon, oxygen, and nitrogen (78.6% C, 14.3% O, 7.1% N). This convergence on the easier class is consistent with the higher validity rates: the Non-Mutagen class has more uniform structural patterns that the generator can capture.

On PROTEINS, both models generate secondary-structure distributions built mainly from helix and sheet nodes, but they differ sharply by class. The 1-2-GNN class 0 generator is the closest match to the real Non-Enzyme distribution, whereas class 1 is much less faithful and becomes strongly sheet-heavy. The 1-GNN produces high-quality Non-Enzyme explanations but struggles with Enzyme validity (37%), while the 1-2-GNN achieves high validity on both classes (100% and 94%) but with lower overall fidelity on class 1 ($s = 0.151$). This suggests the two architectures focus on different structural aspects of protein graphs—the 1-GNN’s node-level features capture Non-Enzyme patterns effectively, while the 1-2-GNN’s pairwise features enable broader Enzyme generation but with less embedding-space alignment.

6.3 Local Structure vs. Population Fidelity

The generated-dataset comparison reveals an important distinction between *local structural realism* and *population-level fidelity*. Across both datasets, the generators preserve degree distributions much better than graph-size distributions. This means that the local connectivity pattern of a typical generated graph often looks plausible even when the overall population of generated graphs is systematically shifted toward larger or more regular structures.

This pattern is clearest on MUTAG, where all four generators remain oversized despite matching average degree closely. The strongest positive case is 1-2-GNN on class 1 (Non-Mutagen): its node-type proportions and graph sizes are closer to the real class than the corresponding 1-GNN generator. However, this advantage is not universal. On MUTAG class 0, the 1-2-GNN overproduces halogens and diverges more strongly from the real atom distribution than the 1-GNN. On PROTEINS, 1-2-GNN class 0 is the cleanest structural match, while 1-2-GNN class 1 is substantially less faithful than its class-0 counterpart. The most defensible interpretation is therefore not that 1-2-GNN is uniformly “better”, but that the two architectures produce different prototype families and that the higher-order model yields the clearest fidelity gain only in selected cases.

6.4 The Granularity Problem

Most generated explanations have granularity $\kappa \approx 0$, meaning they remain close to full-graph prototypes rather than small substructures. This is a structural limitation of the GIN-Graph approach: the generator produces graphs of a fixed maximum size N , and the Gumbel-Softmax sampling tends to activate many nodes. For MUTAG ($N = 28$, average graph ~ 18 nodes), the generated populations still average 23–27 active nodes. For PROTEINS ($N = 50$, average graph ~ 26 nodes), the class 0 generators remain near the size cap (46.6 nodes for 1-GNN and 50.0 for 1-2-GNN), while the class 1 generators are somewhat smaller (24.0 and 29.5 nodes) but still do not behave like sparse subgraph explanations.

Model-level explanations are inherently coarse-grained: they answer “what does a typical class member look like?” rather than “which substructure drives the prediction?” This is a property of the method, not a flaw—the two questions require different explanation approaches.

6.5 Limitations

Several limitations qualify our findings:

Single seed, no error bars. All experiments use a single data split (seed 42) without cross-validation. With only 38 test graphs on MUTAG, the identical 89.5% best accuracy for both models could be coincidence—a difference of one misclassified graph changes accuracy by 2.6 percentage points. Similarly, the validity differences on MUTAG (82% vs. 78%, i.e. 4 graphs out of 100) may not be statistically significant without variance estimates across seeds, which is why we focus on the *qualitative* divergence in generated explanations rather than these marginal quantitative differences. The cross-model classification results (9.4%, 12.4% agreement) are extreme enough to be meaningful even without error bars. The PROTEINS results are more robust (224 test graphs), but the complementary class-specific pattern could also reflect data split artefacts.

No cross-validation. Best test accuracy is reported as the maximum over epochs (early stopping by oracle), which overestimates generalisation performance. A more rigorous evaluation would use a held-out validation set for model selection.

Classifier–generator coupling. GIN-Graph explanations are shaped by how the classifier’s internal representations interact with the generator’s optimisation landscape. The 1-2-GNN class 0 (Non-Enzyme) generator on PROTEINS achieves moderate prediction probability ($p = 0.564$), lower than the 1-GNN’s near-perfect $p = 0.986$ on the same class. We attribute this gap to an *embedding dimensionality mismatch*: the 1-2-GNN classifier operates on a concatenated embedding $\mathbf{h}_G = [\mathbf{h}_G^{(1)} \parallel \mathbf{h}_G^{(2)}] \in \mathbb{R}^{128}$, double the 1-GNN’s $\mathbb{R}^{64}$, while the generator uses the same architecture in both cases. The 2-GNN component’s response to adjacency changes is mediated by pair-level aggregation (Equations 27–28), creating a more complex gradient landscape. However, with graph size and node type regularisation, the generator achieves sufficient prediction probability for meaningful cross-model analysis (98.4% same-model agreement).

The 1-GNN class 1 (Enzyme) generator (37% validity, $s = 0.356$) exhibits a different pattern: $p = 0.919$ indicates the generator can find the target class region, but the generated graphs diverge from the real Enzyme embedding centroid. This is a fidelity problem (the generated graphs are atypical Enzyme structures) rather than a guidance problem (the classifier does recognise them as Enzyme).

Generator–classifier output space mismatch. A fundamental consideration is that GIN-Graph was designed for standard (1-WL) GNNs, and its generator architecture reflects this. The generator outputs an adjacency matrix $\hat{A}$ and node feature matrix $\hat{X}$ —quantities that a 1-GNN directly consumes. A 1-GNN’s decisions are based on node features and local neighbourhoods, which the generator can target directly: “place nitrogen here, connect it to these carbons.” The 1-2-GNN, however, makes decisions in a pairwise feature space derived from all $\binom{n}{2}$ node pairs. The generator has no mechanism to directly control pair-level representations—it can only influence them indirectly through edge and node choices. Despite this, the generator achieves high same-model agreement for all

configurations ($\geq 94\%$) after structural regularisation, demonstrating that the indirect optimisation path is viable. However, the consistently lower prediction probability for the 1-2-GNN ($p = 0.564$ vs. 0.986 on PROTEINS class 0) suggests that a higher-order-aware generator—one that directly produces pair-level features—could improve results further.

Evaluation metrics. The validation score $v = (s \cdot p \cdot d)^{1/3}$ can be dominated by a single low component. The degree score, which measures structural realism via average degree alone, is a coarse proxy that does not capture finer structural properties like motif frequencies or degree distributions.

No baselines. We do not compare with instance-level explanation methods (GNNEExplainer, PGExplainer) or other model-level approaches. GIN-Graph is the only model-level method we are aware of that generates full graphs via GAN, so direct model-level baselines are limited. However, comparing the *utility* of model-level vs. instance-level explanations on the same classifiers would contextualise our findings.

6.6 Future Work

Several directions could strengthen this investigation:

- **Higher-order-aware generator.** The most impactful extension would be a generator architecture designed for k -GNN classifiers. Rather than outputting only adjacency and node features, such a generator could directly produce pair-level (or k -set-level) features, or use higher-order message passing internally so that it “thinks” in the same representation space as the classifier. This would disentangle the question of whether the 1-2-GNN learns richer structural patterns from the confound of generator-classifier mismatch identified in our limitations.
- Use cross-validation and multiple random seeds for robust accuracy and explanation quality estimates. The current single-seed results, while consistent across configurations, may not generalise.
- Investigate *why* the 1-GNN and 1-2-GNN learn different class representations—e.g., which specific pairwise patterns drive the 1-2-GNN’s more selective Non-Enzyme boundary—using attention or gradient-based attribution on the 2-GNN component.
- Extend to the 1-2-3-GNN architecture, which adds 3-set (triplet) message passing for even higher expressiveness, to test whether the interpretability benefit continues up the k -WL hierarchy.
- Incorporate richer structural evaluation metrics beyond average degree, such as clustering coefficients, motif frequencies, or subgraph counts, to better capture the qualitative differences between generated explanations.
- Apply the framework to larger and more diverse datasets where the expressiveness gap between 1-WL and higher-order tests is more pronounced.

7 Conclusion

We compared a standard 1-GNN with a hierarchical 1-2-GNN for model-level explanation generation using the GIN-Graph framework, with all configurations fully trained for 300 epochs across both datasets and both classes.

Our central finding is that the two architectures learn *qualitatively different internal repre-*

sentations of graph classes. On MUTAG, both models achieve identical classification accuracy (89.5%), yet they generate strikingly different explanations: the 1-GNN produces more N/O-rich mutagenic structures, while the 1-2-GNN favours more strongly halogenated compounds. This divergence—same accuracy, different explanations—provides the cleanest evidence that higher-order message passing leads the model to encode different structural features, not merely “better” ones.

Cross-model classification of 500 generated graphs per class confirms this divergence quantitatively. On MUTAG, the 1-2-GNN’s Non-Mutagen graphs are rejected by the 1-GNN (12.4% agreement). On PROTEINS, 1-GNN-generated Non-Enzyme graphs are rejected by the 1-2-GNN (9.4% agreement), while the 1-2-GNN’s Non-Enzyme graphs are accepted by both models (98.4%/100%). These asymmetries reveal architecture-specific decision boundaries, with the clearest selective advantage for the 1-2-GNN appearing on PROTEINS Non-Enzyme and MUTAG Non-Mutagen rather than uniformly across all settings.

At the same time, the generated-dataset comparison shows that the generators preserve local degree structure more reliably than graph-size distributions, and the embedding-space analysis shows that generated graphs often occupy separate prototype manifolds rather than the full real-data cloud. The most defensible conclusion is therefore nuanced: higher-order message passing changes what kinds of class prototypes are learned, and in selected cases improves fidelity, but it does not produce a blanket advantage on every class or every dataset.

Our fully differentiable dense wrapper enables GIN-Graph explanation generation for hierarchical k -GNN architectures while maintaining gradient flow through both node features and adjacency matrices. Additionally, our corrected validation metric—computing actual cosine similarity between generated and real graph embeddings rather than using prediction probability as a proxy—provides a more faithful measure of explanation quality. These technical contributions open the door to studying interpretability across the k -WL expressiveness hierarchy.

References

- [Borgwardt et al.(2005)] K. M. Borgwardt, C. S. Ong, S. Schönauer, S. V. N. Vishwanathan, A. J. Smola, and H.-P. Kriegel. Protein function prediction via graph kernels. *Bioinformatics*, 21(suppl_1):i47–i56, 2005.
- [Cai et al.(1992)] J.-Y. Cai, M. Fürer, and N. Immerman. An optimal lower bound on the number of variables for graph identification. *Combinatorica*, 12(4):389–410, 1992.
- [Debnath et al.(1991)] A. K. Debnath, R. L. Lopez de Compadre, G. Debnath, A. J. Shusterman, and C. Hansch. Structure-activity relationship of mutagenic aromatic and heteroaromatic nitro compounds. *J. Med. Chem.*, 34(2):786–797, 1991.
- [Gulrajani et al.(2017)] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville. Improved training of Wasserstein GANs. In *NeurIPS*, 2017.
- [Jang et al.(2017)] E. Jang, S. Gu, and B. Poole. Categorical reparameterization with Gumbel-softmax. In *ICLR*, 2017.

- [Morris et al.(2019)] C. Morris, M. Ritzert, M. Fey, W. L. Hamilton, J. E. Lenssen, G. Rattan, and M. Grohe. Weisfeiler and Leman go neural: Higher-order graph neural networks. In *AAAI*, 2019.
- [Sun et al.(2025)] J. Sun, S. Pei, F. Zhang, and N. V. Chawla. GIN-Graph: A generative interpretation network for model-level explanation of graph neural networks. *Neurocomputing*, 2025.